

Generative Image Models

Part 1

From VAEs, GANs and U-Net to the cutting edge of Diffusion models, exploring the architectures that redefined digital media world.



About me

Physicist turned to Data Scientist

PhD at **EPFL**

10+ year in Data Science / Data Engineering

Co-Founder of  **bigblue.ai**

Data Science & Entrepreneurship



Dr. Dimitris Mousadakos

Table of Contents

01

Motivation

02

Autoencoders

03

Variable Autoencoders

04

Generative Adversarial Networks

05

U-Net



- Learn what it actually means for a computer to "imagine" and create an entirely new, realistic image from scratch rather than just copying existing ones.
- **VAEs:** Learn the first building blocks for image/data generation
- **GANs:** Discover how two networks can train each other by playing a game of cat-and-mouse, resulting in highly realistic fake images.
- **U-Net:** Explore the underlying blueprint that allows AI to understand both the broad context of a picture and its tiniest pixel-level details.
- **Diffusion:** Uncover the breakthrough method behind modern tools like Midjourney and DALL-E, which build stunning images by slowly clearing away random visual static.

Motivation: Why Generative Models?

- **Solving the Data Bottleneck:** Generating privacy-compliant, synthetic data to train other AI systems (e.g., rare diseases in medical imaging, edge cases in autonomous driving).
- **Augmenting Human Creativity:** Revolutionizing industries like gaming, cinema, fashion, marketing and industrial design by turning text prompts or rough sketches into high-fidelity prototypes in seconds.
- **Advanced Image Manipulation:** Powering everyday tools for image restoration, super-resolution upscaling, and editing and inpainting.



Discriminative vs. Generative

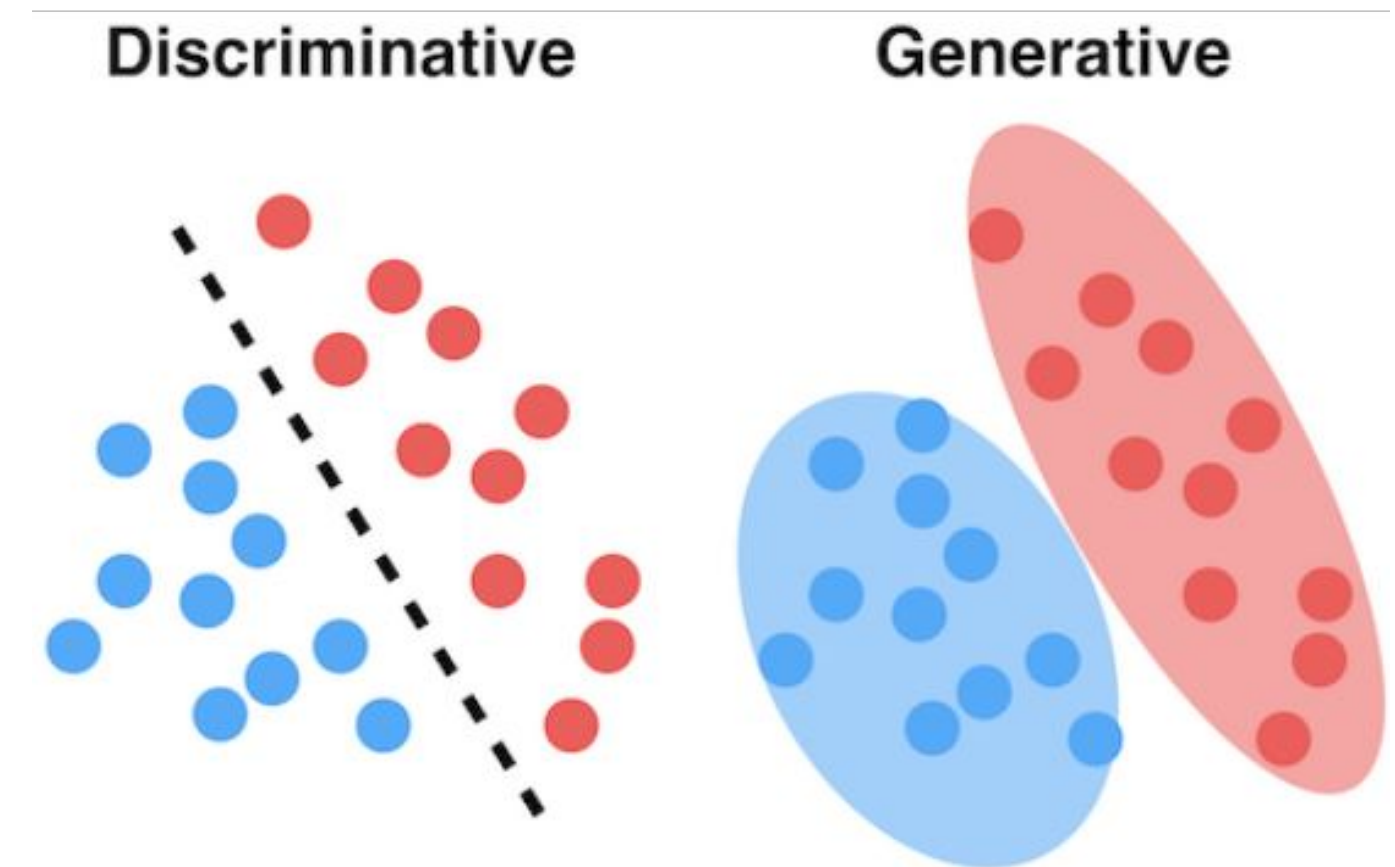
Discriminative Models

Given an image X , predict a label Y . Estimates $P(Y | X)$.

Limitation: Cannot model or sample from $P(X)$, meaning it cannot generate new data. (e.g., Is this a cat or a dog?)

Generative Models

Models $P(X)$ directly. Allows sampling of new images from the learned distribution. (e.g., Create a new picture of a cat).



The Generative Timeline

2013 | Variational Autoencoders (VAEs)

Introduced the concept of mapping data into probabilistic latent spaces.

2014 | GANs Invented (Goodfellow)

Established the adversarial minimax game for sharp image synthesis.

2015 | U-Net Created

Introduced for biomedical segmentation; later becomes the ultimate denoising engine.

2018 | StyleGAN (NVIDIA)

Revolutionized high-fidelity face generation with disentangled style controls.

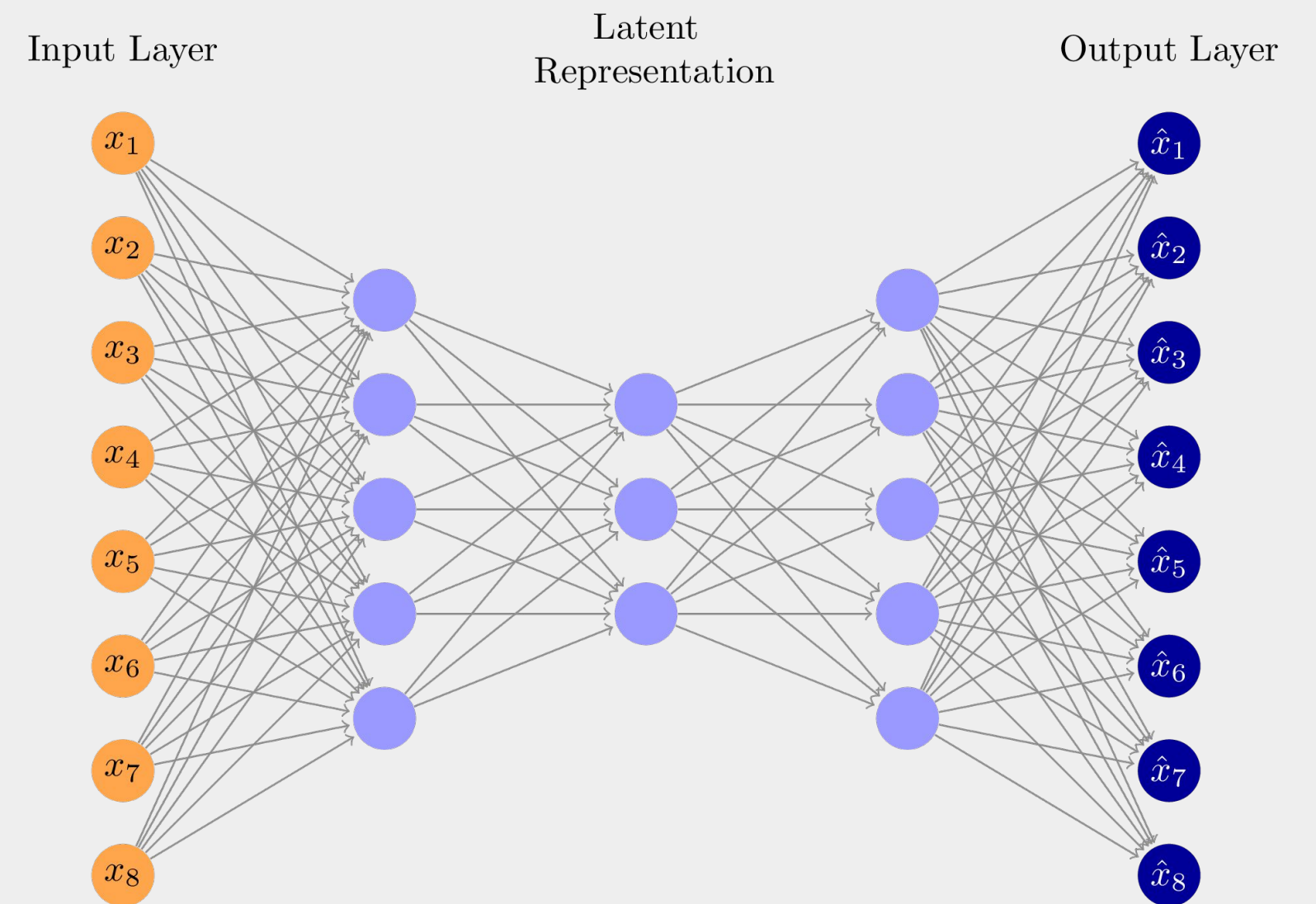
2020 | Denoising Diffusion Probabilistic Models (DDPM) Breakthrough

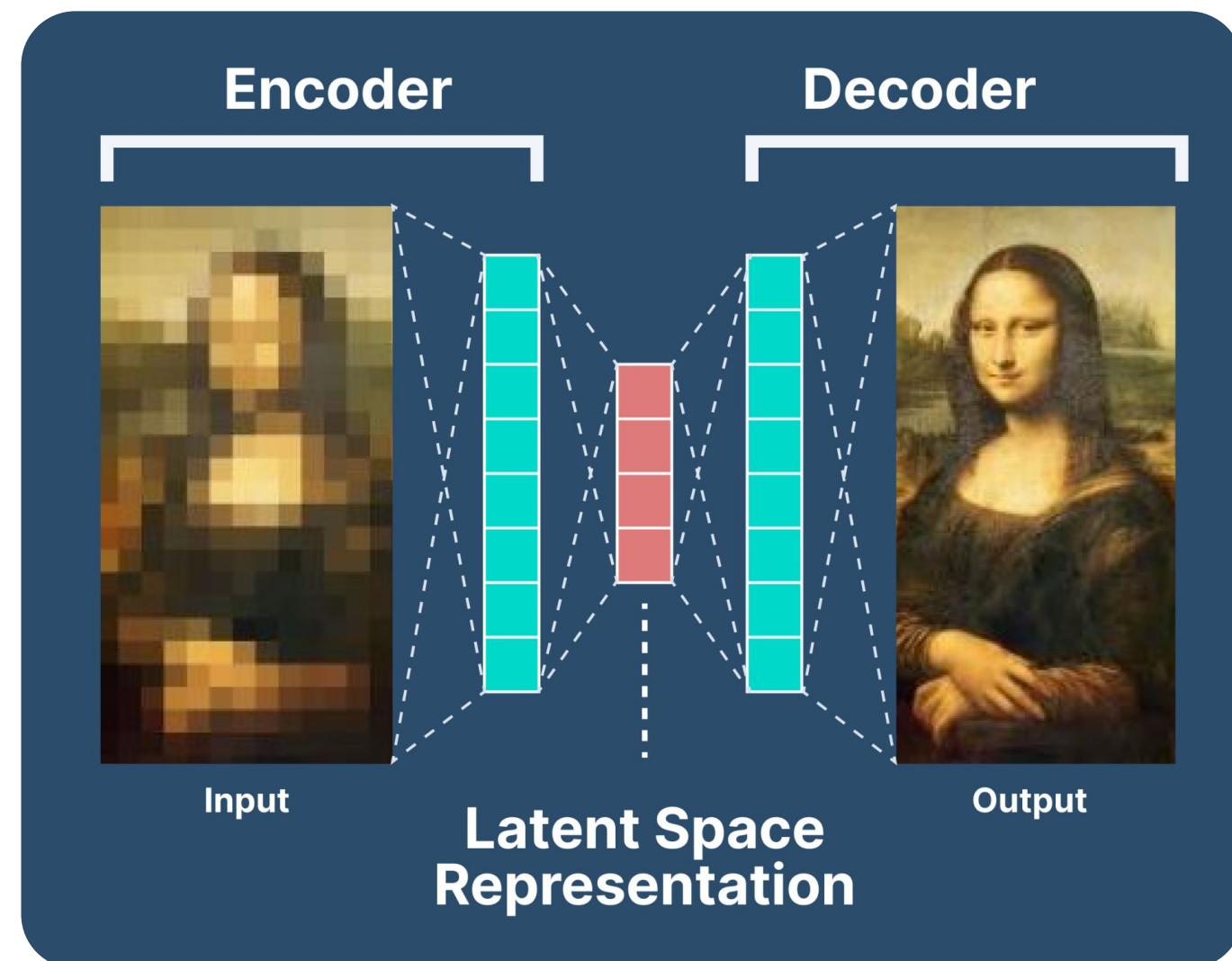
Connected thermodynamics theory to neural networks, making diffusion practical.

2022 | Latent/Stable Diffusion

Combined VAEs with Diffusion, enabling ultra-high-resolution generation on consumer hardware.

Autoencoders





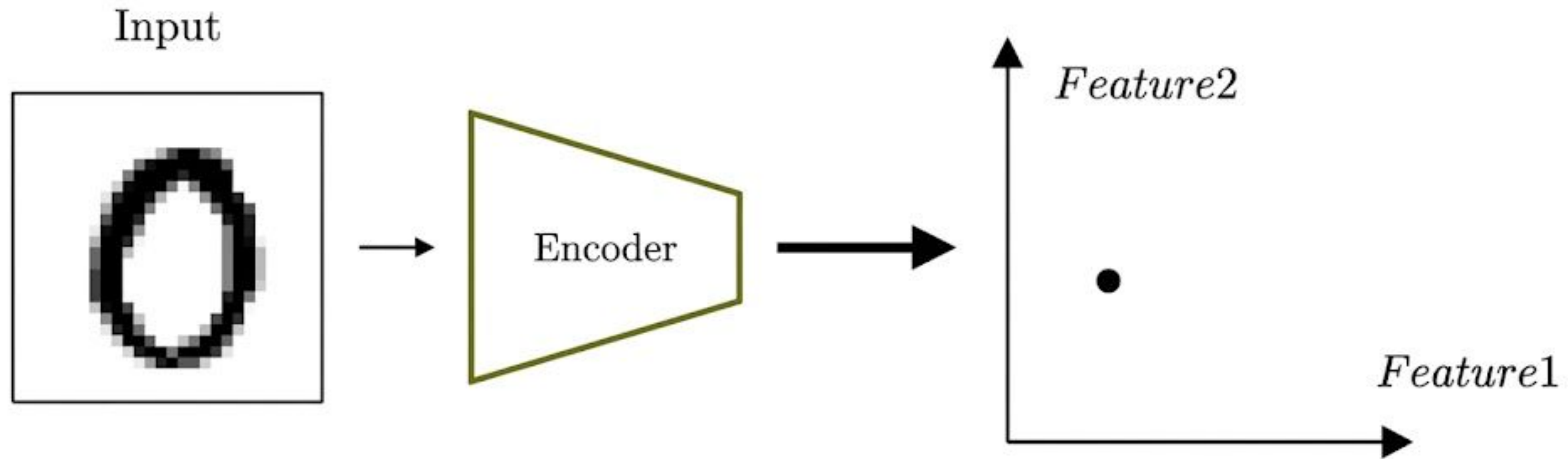
The architecture typically consists of three main components arranged in a symmetrical "hourglass"

Encoder: This part of the network takes high-dimensional input (like an image) and compresses it.

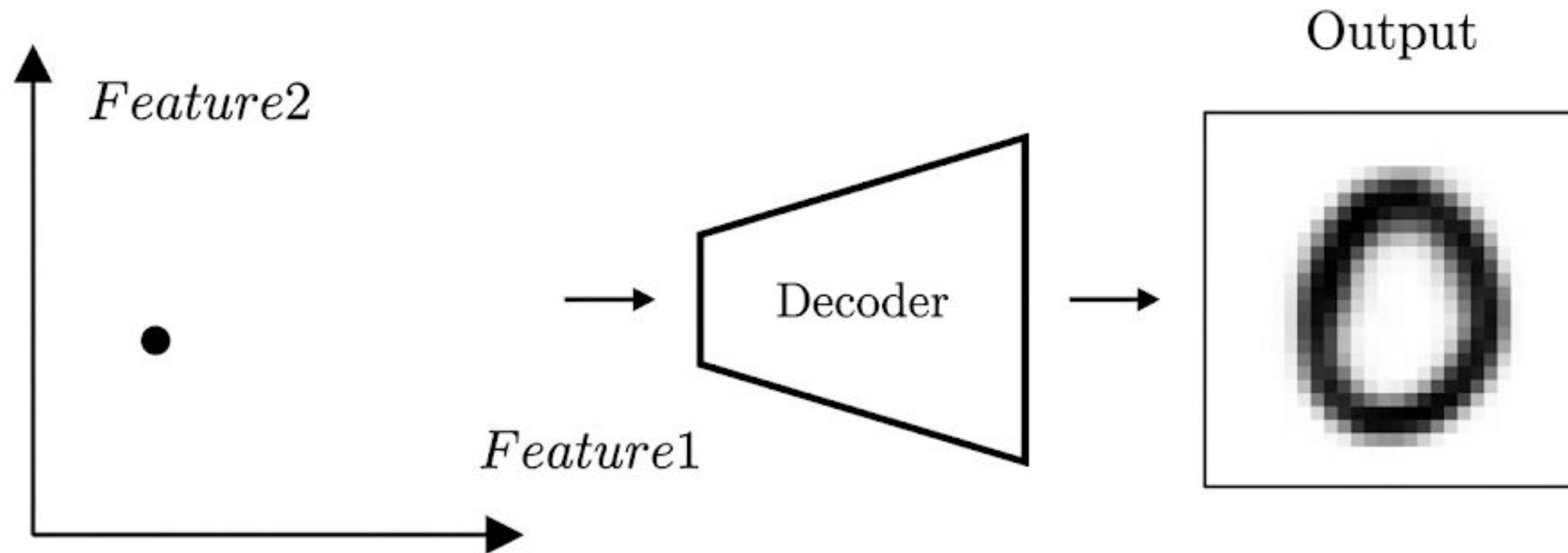
Bottleneck (Latent Space): This is the narrowest part of the network. It forces the data into a lower-dimensional "code" or "embedding". Because this layer is so small, the network cannot simply "memorize" the input; it must prioritize the most essential features to fit through the gap.

Decoder: This section takes the compressed code and attempts to expand it back into the original input as accurately as possible.

Autoencoders



Autoencoders



The Learning Objective & the Bridge to Generation

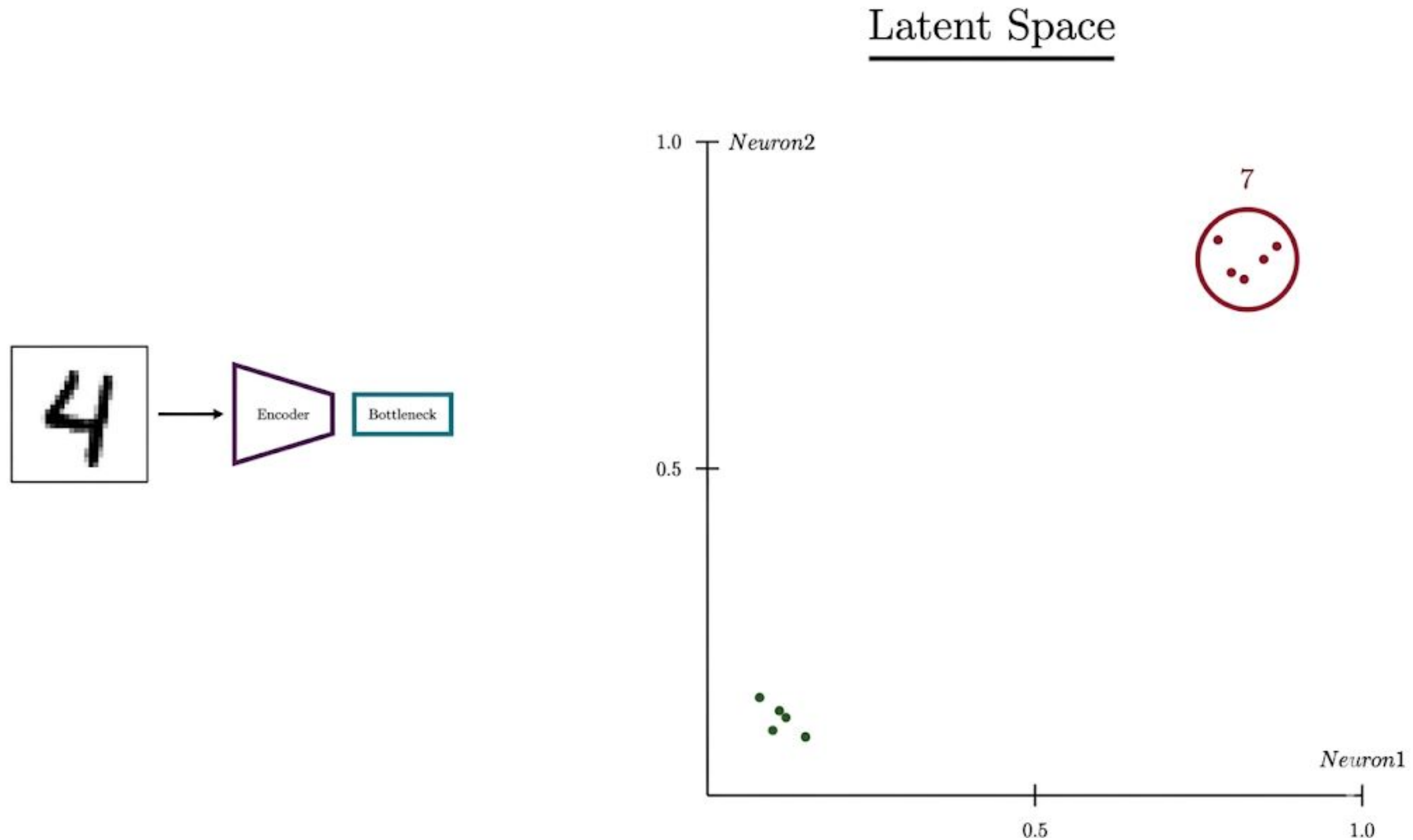
For the network to learn anything useful, we have to measure how well the Decoder's output y matches the Encoder's input x . We do this by minimizing a reconstruction loss.

This is typically the Mean Squared Error (MSE):

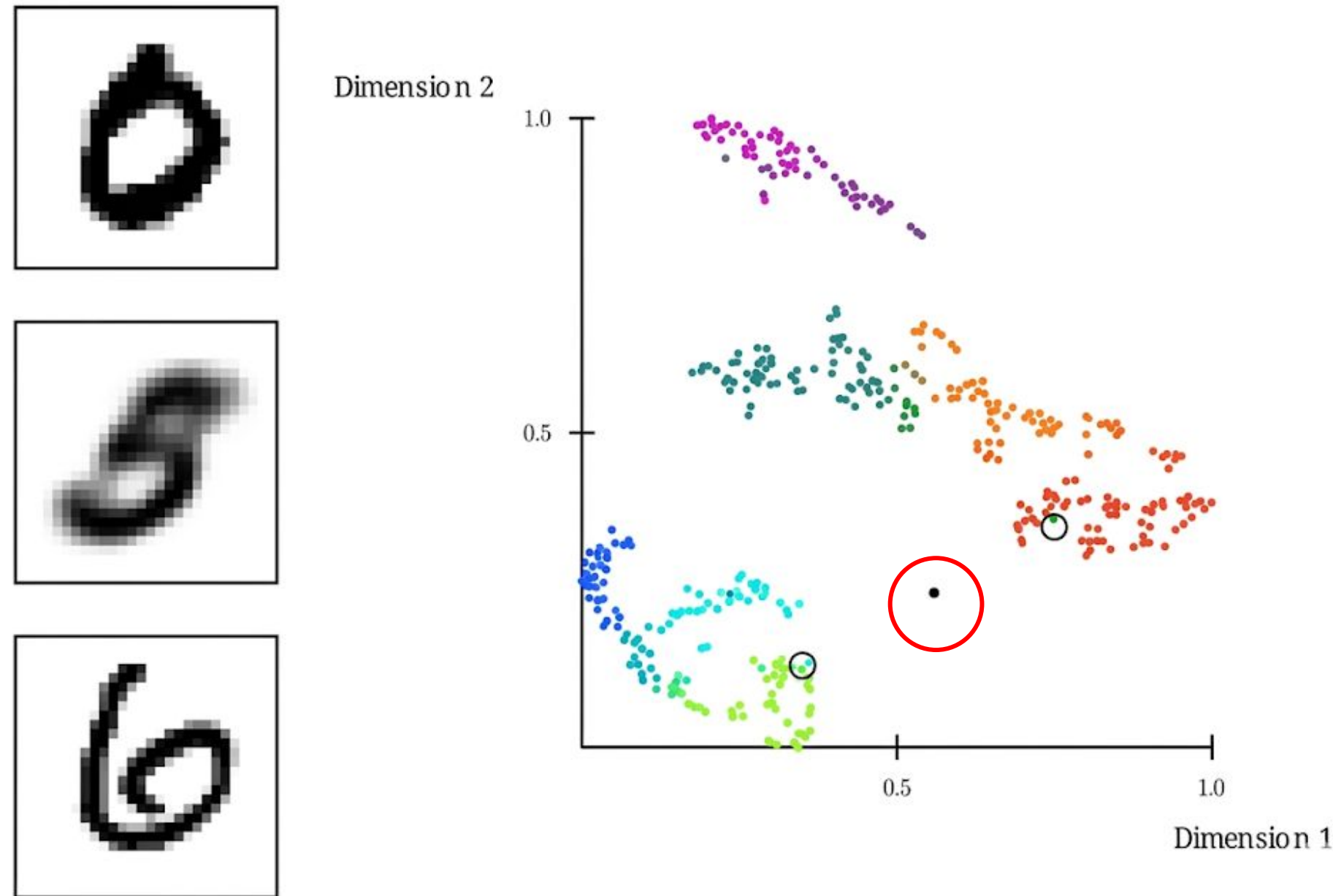
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

By minimizing this loss via backpropagation, the network learns the underlying manifold of the data distribution. It effectively performs non-linear dimensionality reduction (a much more powerful cousin to PCA).

The Learning Objective & the Bridge to Generation

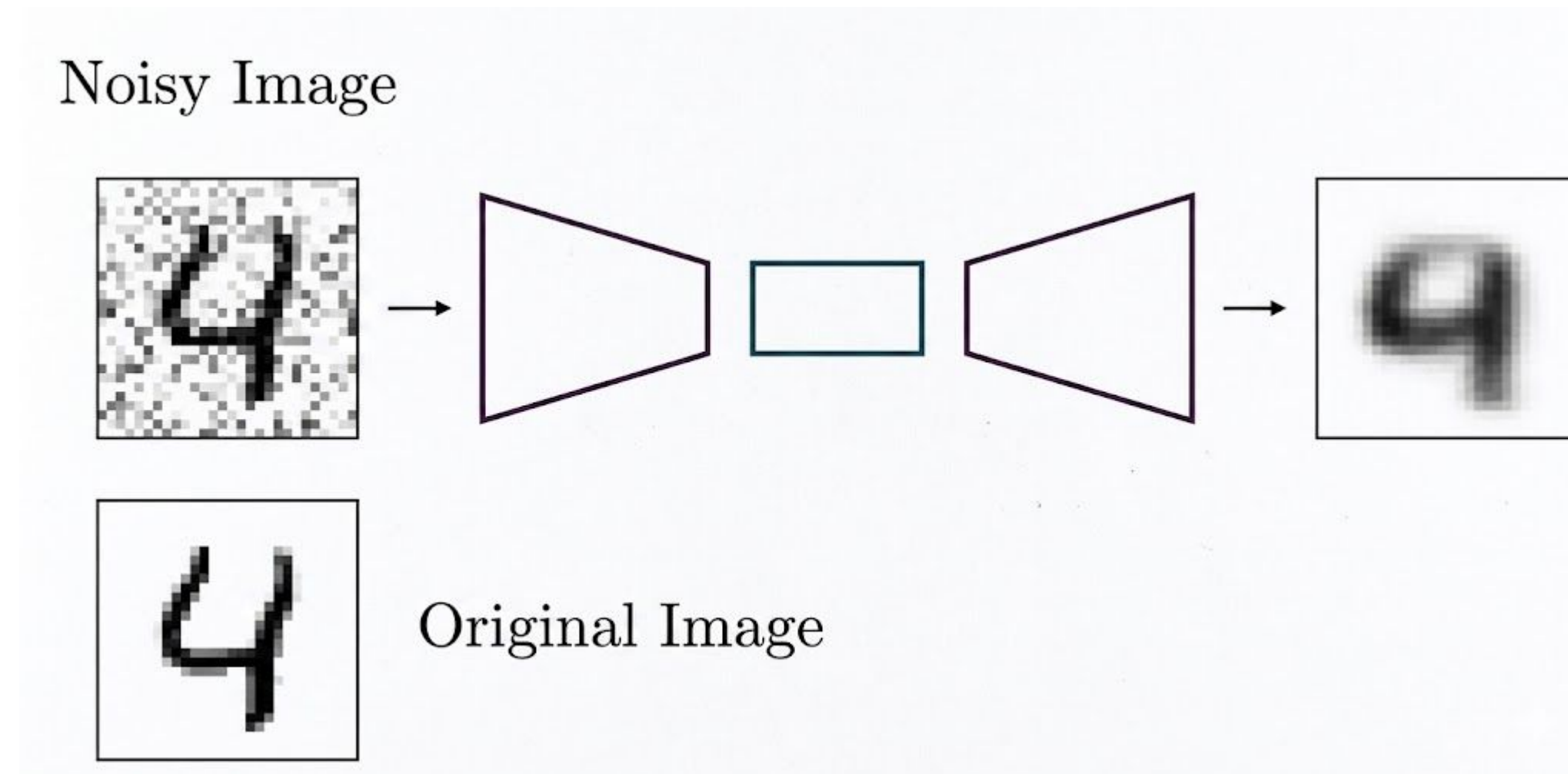


The Learning Objective & the Bridge to Generation



However, autoencoders are not inherently generative. If you pass a picture of a cat into an autoencoder, it will give you that same cat back. If you pick a random point in its latent space and decode it, you will likely get meaningless results.

The Learning Objective & the Bridge to Generation



However, autoencoders are not inherently generative. If you pass a picture of a cat into an autoencoder, it will give you that same cat back. If you pick a random point in its latent space and decode it, you will likely get meaningless results.

Common Types & Applications

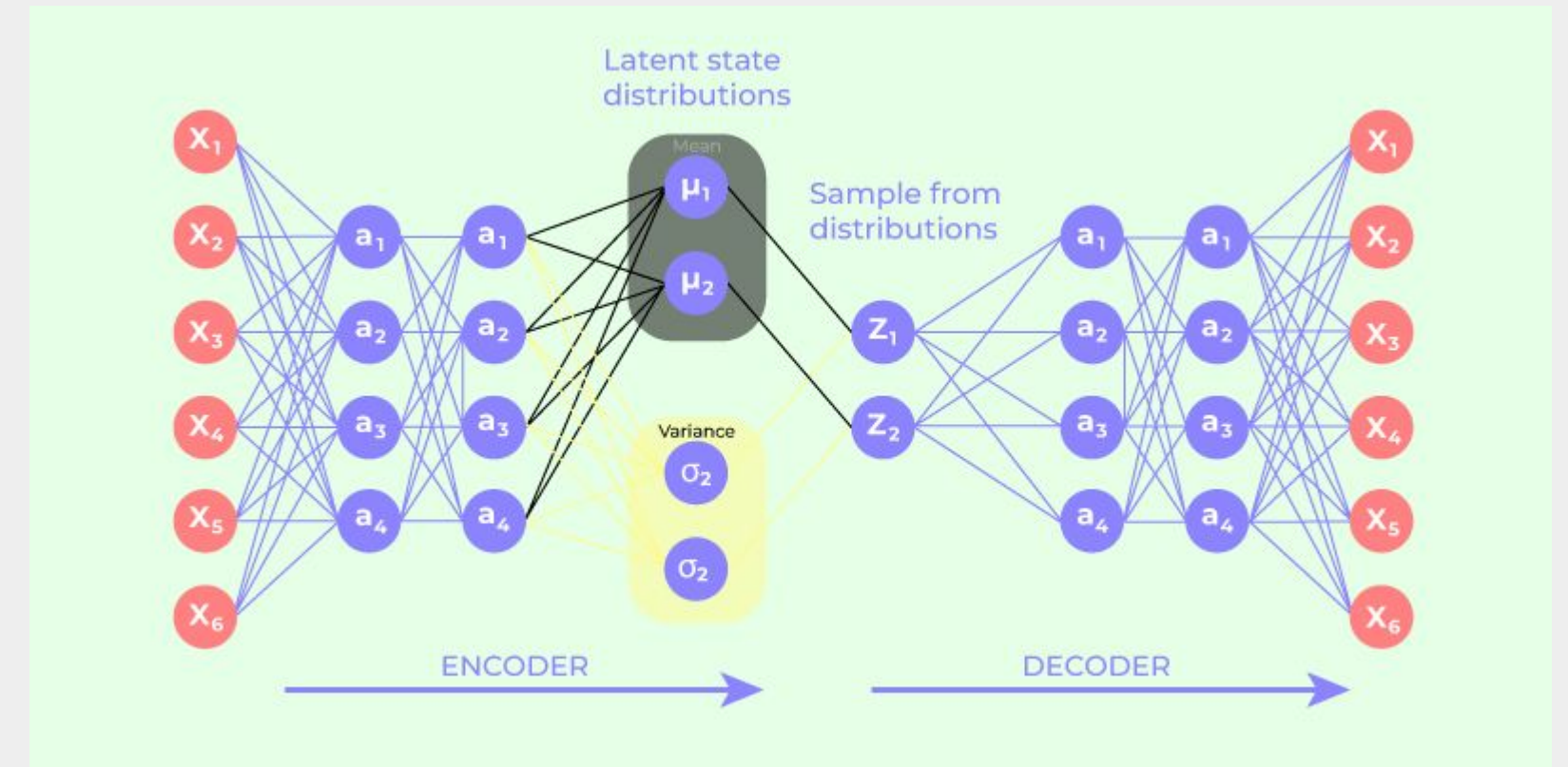
Denoising Autoencoders: Trained by taking "noisy" or corrupted data (like a grainy photo) and teaching the model to output the original "clean" version.

Variational Autoencoders (VAEs): A probabilistic version that learns a continuous "latent space." This allows you to sample new, original points from that space to generate entirely new data (like fake faces).

Anomaly Detection: Because the model learns what "normal" data looks like, it will struggle to reconstruct "abnormal" data (like a fraudulent credit card transaction), resulting in a high reconstruction error that flags the anomaly.

Dimensionality Reduction: Similar to PCA but more powerful, it can find non-linear patterns to represent complex data in just a few variables.

Variable Autoencoders



Variable Autoencoders

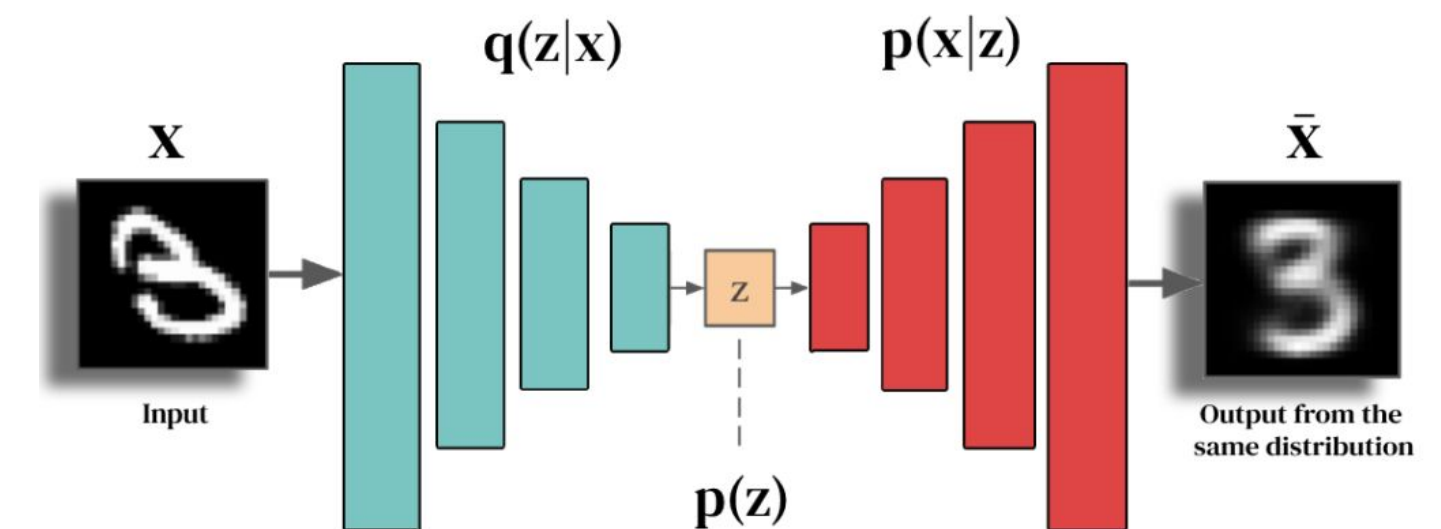
Instead of the encoder mapping an input x to a single deterministic point z , a VAE encoder maps the input to a probability distribution over the latent space.

Specifically, the encoder outputs two vectors for each input: **a mean vector μ and a variance vector σ^2** . These parameters define a multivariate Gaussian distribution.

The actual latent vector z that gets passed to the decoder is then randomly sampled from this specific distribution:

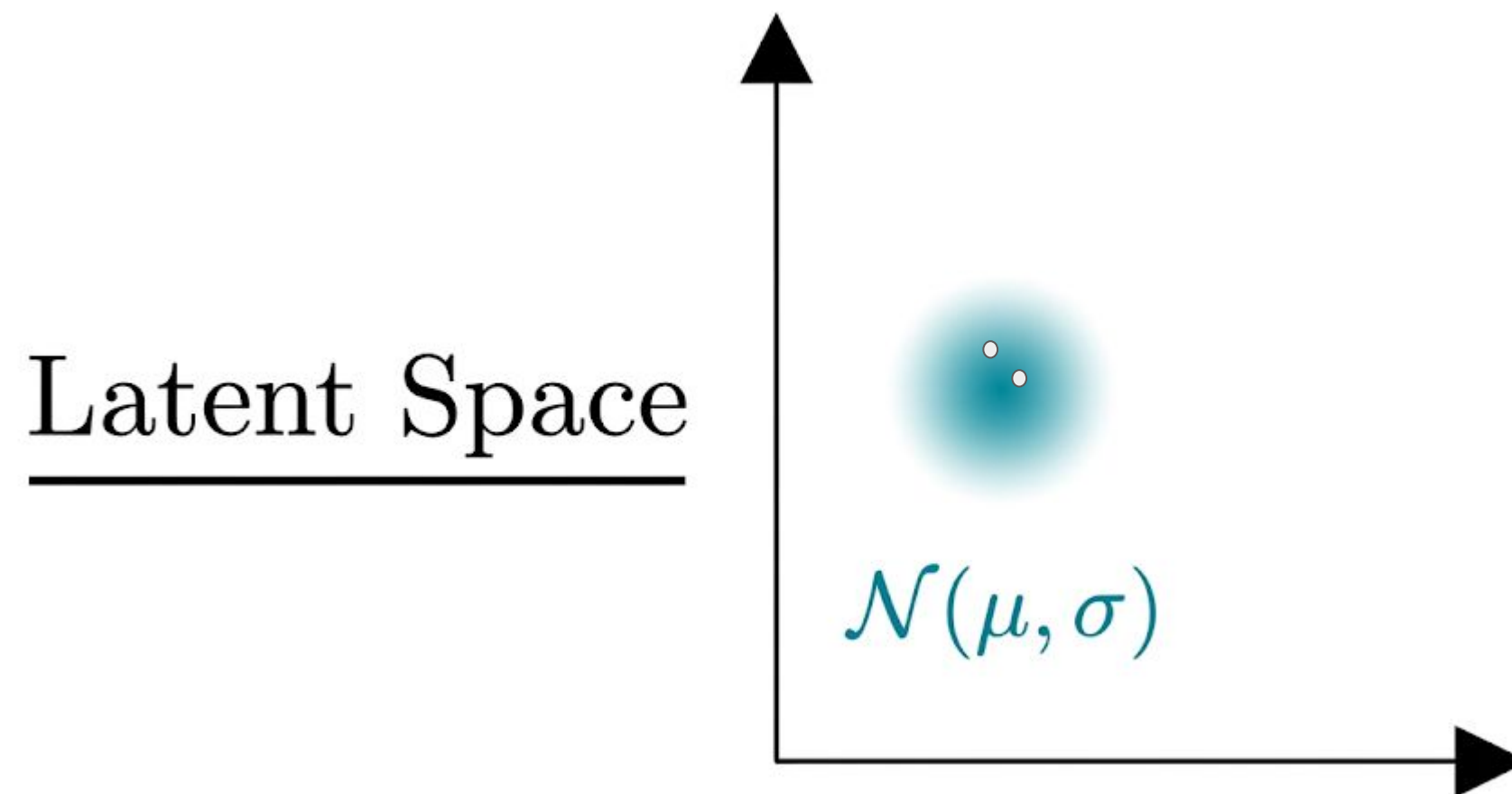
$$z \sim \mathcal{N}(\mu, \sigma^2)$$

By mapping inputs to overlapping distributions rather than isolated points, the VAE enforces continuity. Similar inputs will yield distributions that overlap in the latent space, meaning the decoder must learn to reconstruct plausible outputs from a wider, smoother variety of latent vectors.

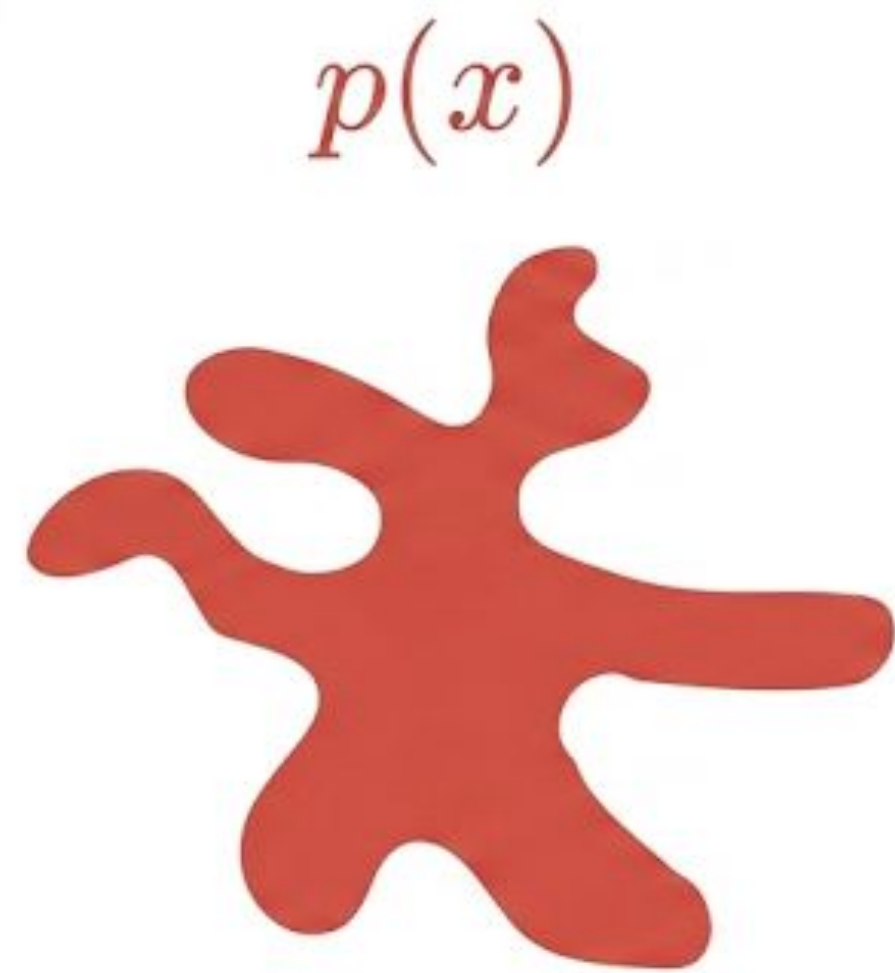


Variable Autoencoders

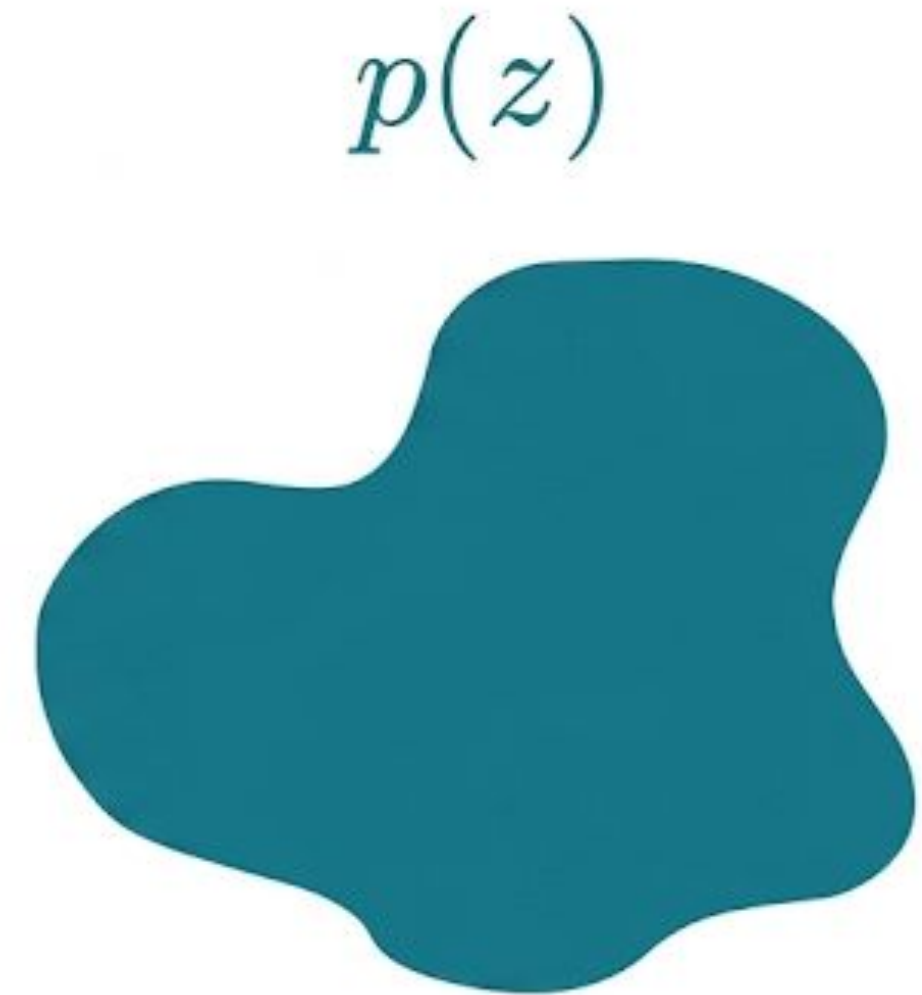
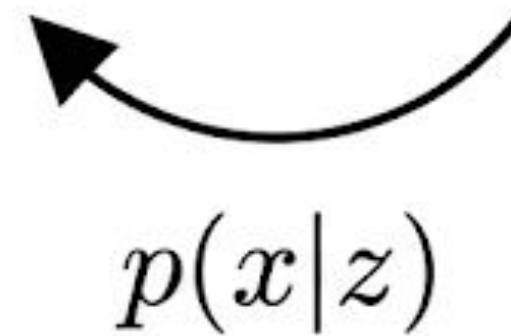
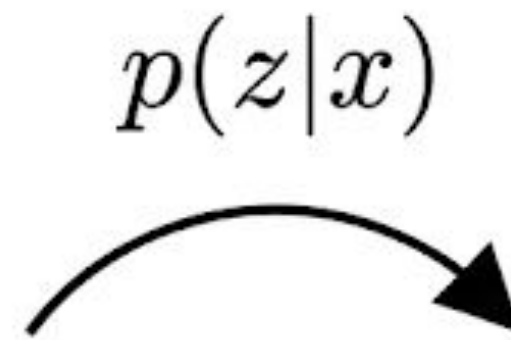
By mapping inputs to overlapping distributions rather than isolated points, the VAE enforces continuity. Similar inputs will yield distributions that overlap in the latent space, meaning the decoder must learn to reconstruct plausible outputs from a wider, smoother variety of latent vectors.



Variable Autoencoders

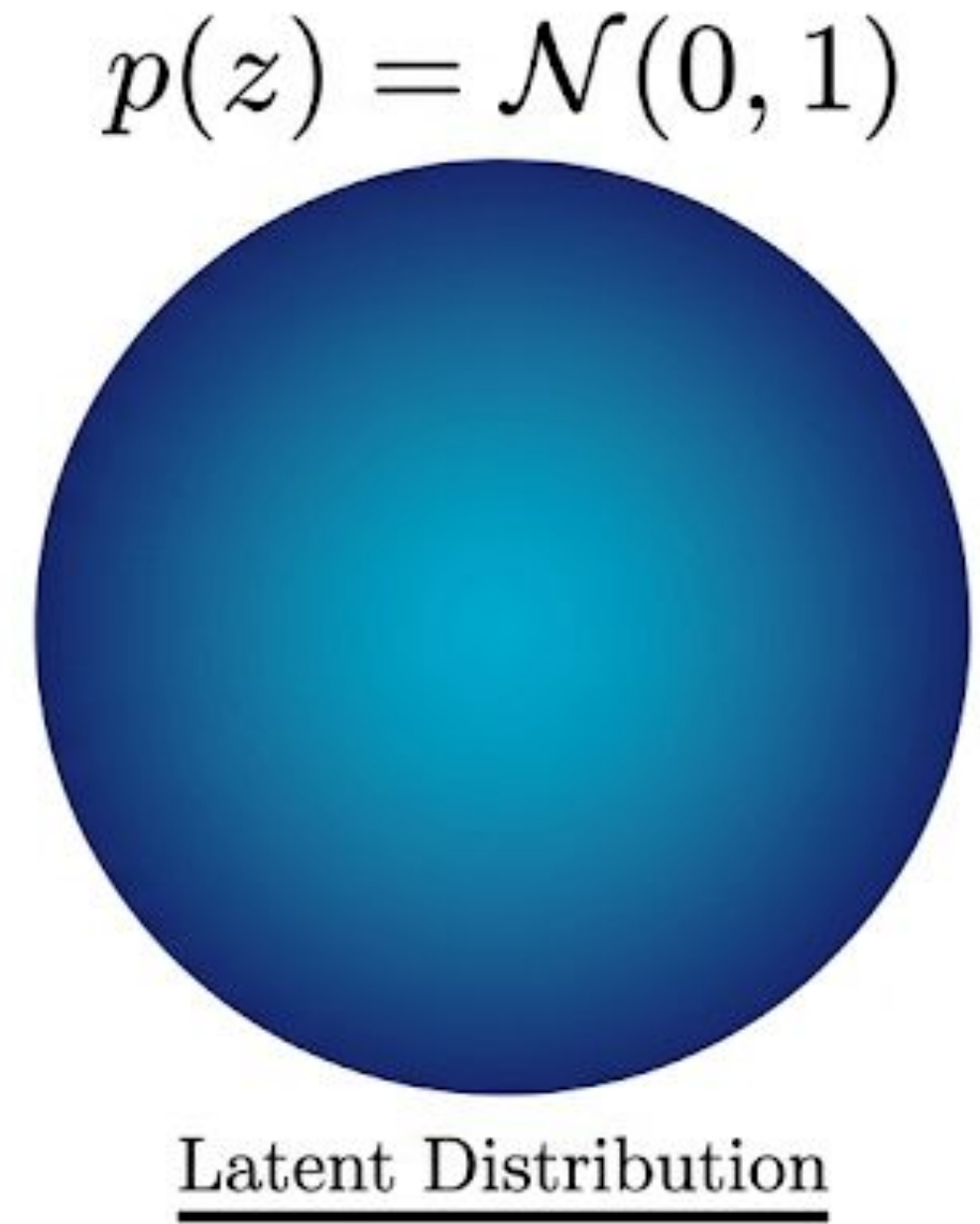


Data Distribution

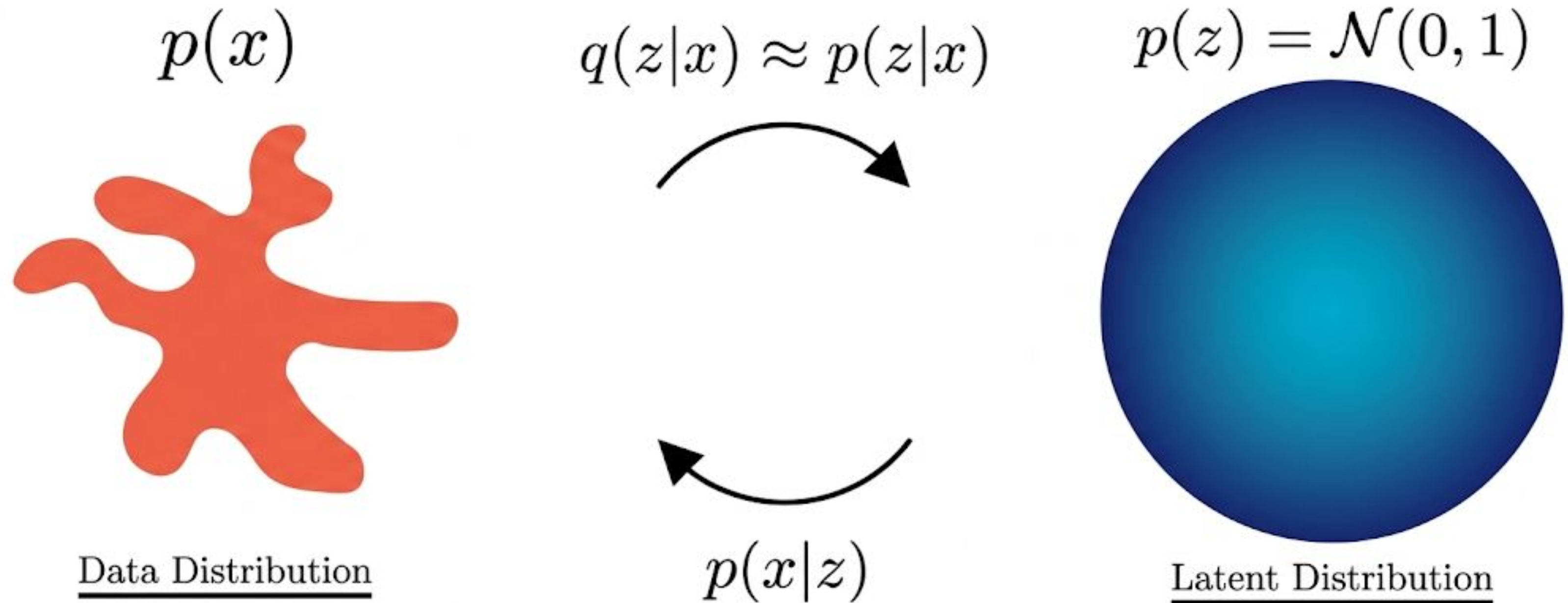


Latent Distribution

Variable Autoencoders



Variable Autoencoders



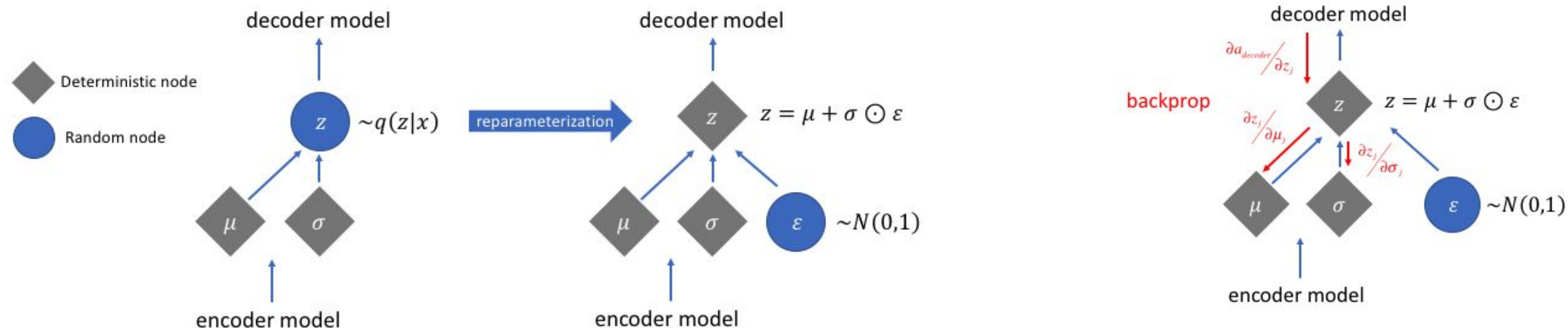
The Reparameterization Trick

Solving the Sampling Problem

Neural networks learn by calculating gradients, but we cannot backpropagate gradients through a stochastic (random) sampling node. The random sampling operation has no derivative.

Instead of sampling z directly from $\mathcal{N}(\mu, \sigma^2)$, we sample a noise variable ϵ from a standard normal distribution $\mathcal{N}(0, I)$. We then calculate z deterministically: $z = \mu + \sigma \odot \epsilon$

Because ϵ is treated as an external input, the gradients can easily flow backward through μ and σ to update the encoder's weights. This elegant mathematical pivot is what makes VAEs trainable.



The Objective Function

The VAE loss function, often called the **Evidence Lower Bound (ELBO)**, is the mechanism that keeps the latent space organized. It consists of two competing mathematical forces:

The Reconstruction Loss: the network must accurately reconstruct the input. This minimizes the difference between x and y , pushing the network to learn meaningful, highly specific distributions for different data points.

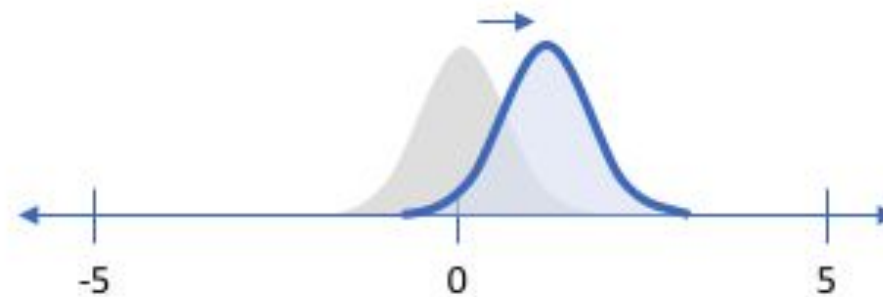
$$\mathbb{E}_q[\log p(x|z)]$$

The Kullback-Leibler (KL) Divergence: If we only used reconstruction loss, the network would cheat. It would shrink the variance σ^2 to zero, turning the distributions back into deterministic points to avoid the noise of sampling. To prevent this, we add a regularization term: the KL divergence between the learned distribution $q(z|x)$ and a standard normal prior $p(z) = \mathcal{N}(0, I)$.

$$\mathcal{L}_{\text{VAE}} = \text{Reconstruction Loss} + \text{KL Divergence}$$

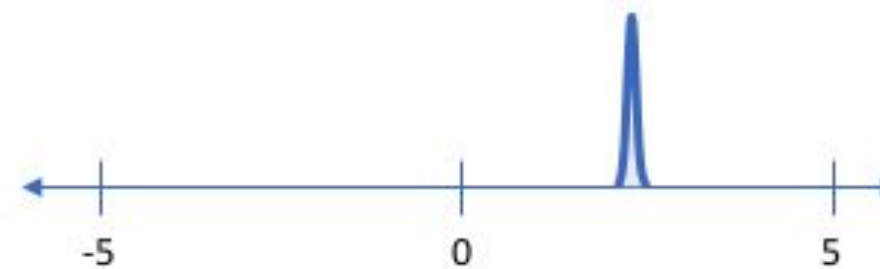
The Objective Function

Penalizing reconstruction loss encourages the distribution to describe the input



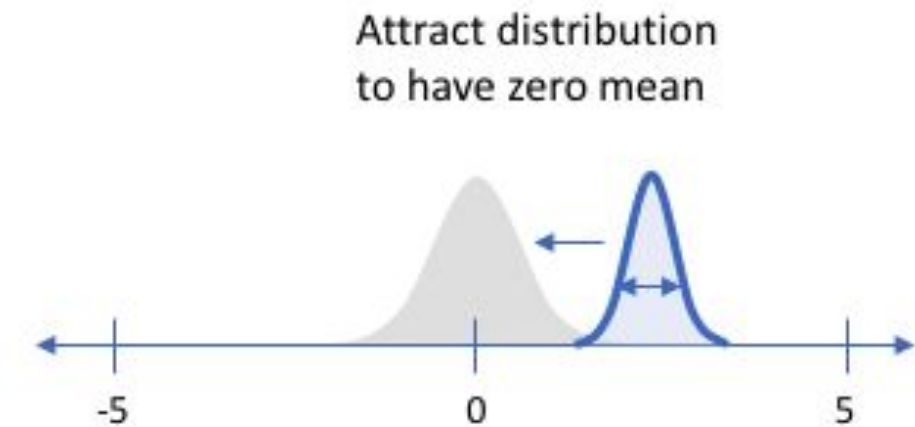
Our distribution deviates from the prior to describe some characteristic of the data

Without regularization, our network can “cheat” by learning narrow distributions



With a small enough variance, this distribution is effectively only representing a single value

Penalizing KL divergence acts as a regularizing force

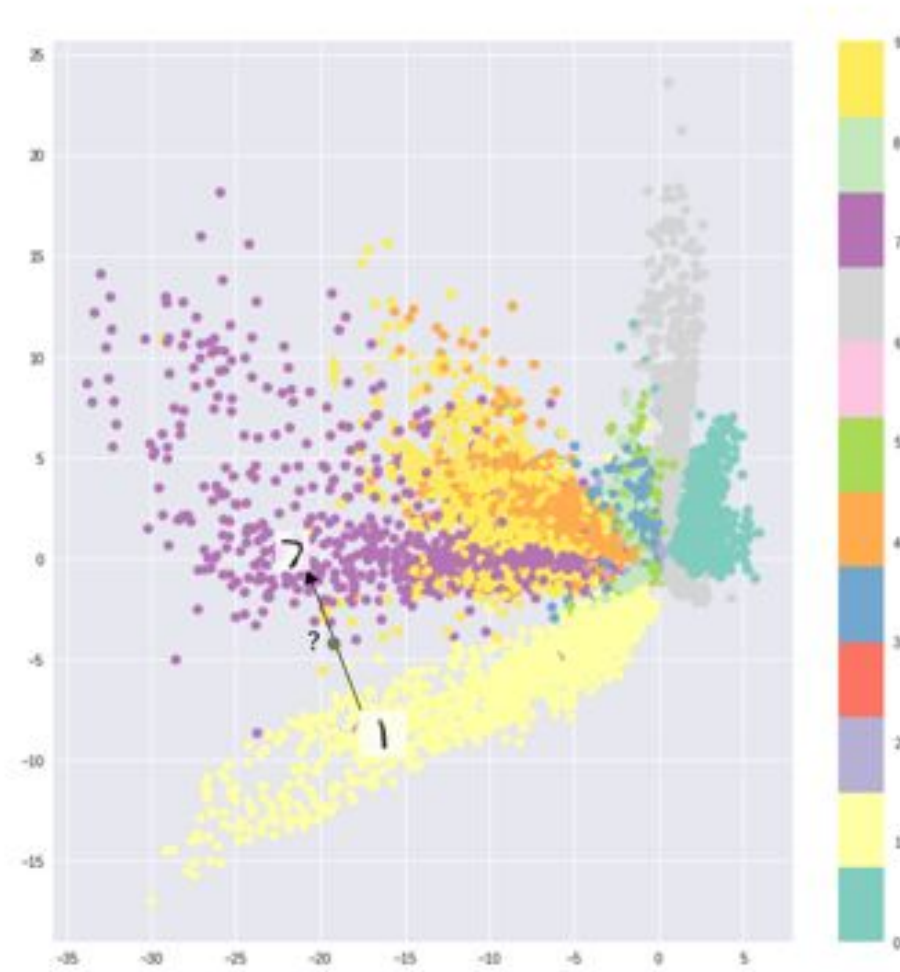


Attract distribution to have zero mean

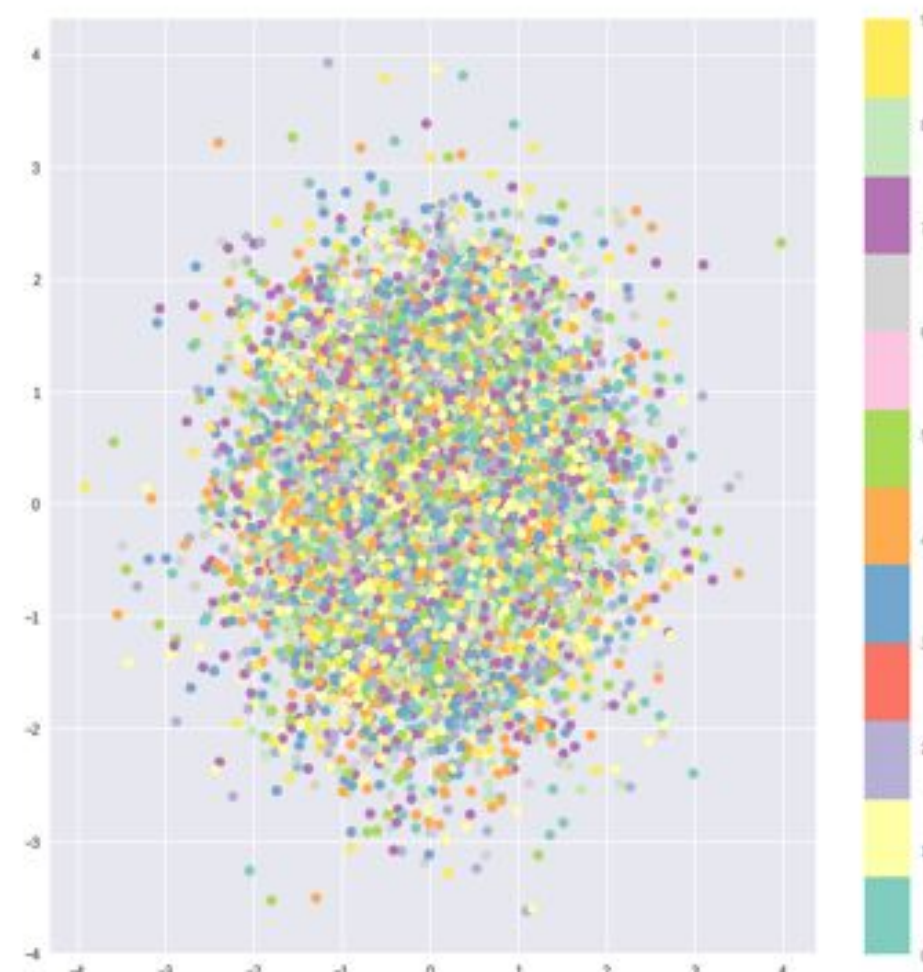
Ensure sufficient variance to yield a smooth latent space

The Objective Function

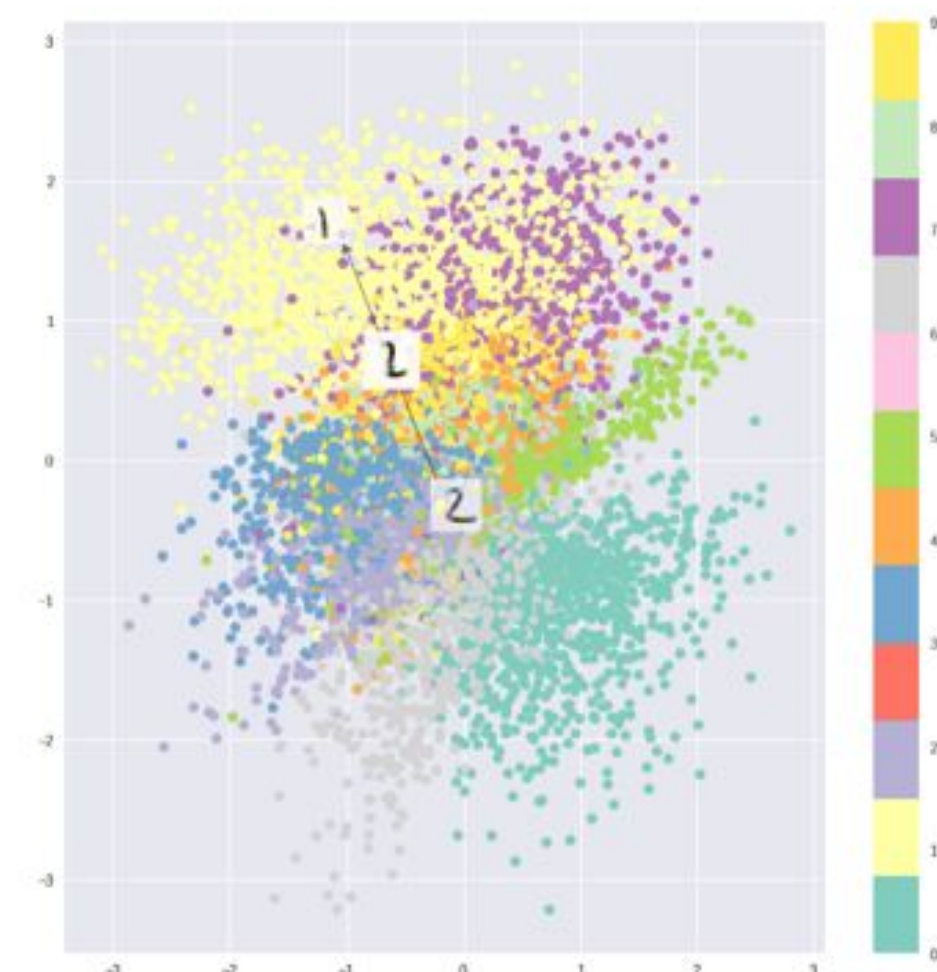
Only reconstruction loss



Only KL divergence



Combination



Because the KL divergence forced the entire latent space to approximate a standard normal distribution during training, we can now entirely discard the encoder.

To generate a brand-new, completely synthetic data point, we simply:

1. Sample a random noise vector z directly from the prior distribution $\mathcal{N}(0, I)$.
2. Pass that vector through the trained decoder.

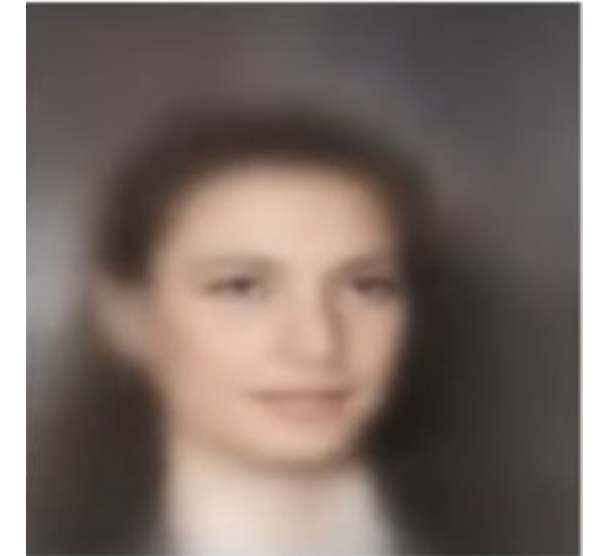
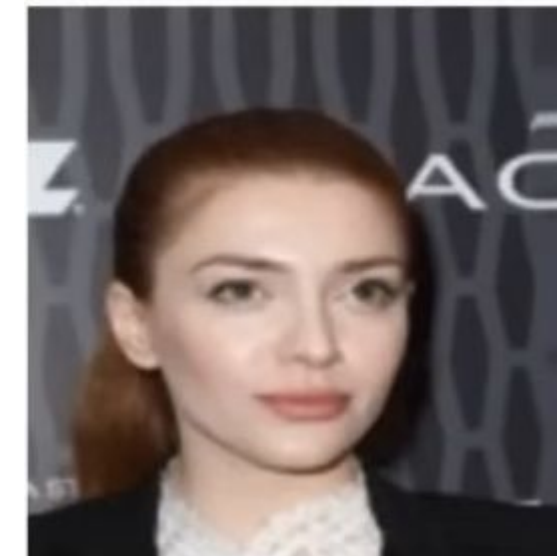
Because the latent space is continuous and complete, the decoder will map that random vector into a highly realistic, novel output that belongs to the original data distribution but never existed in the training set.

Variable Autoencoders Limitations

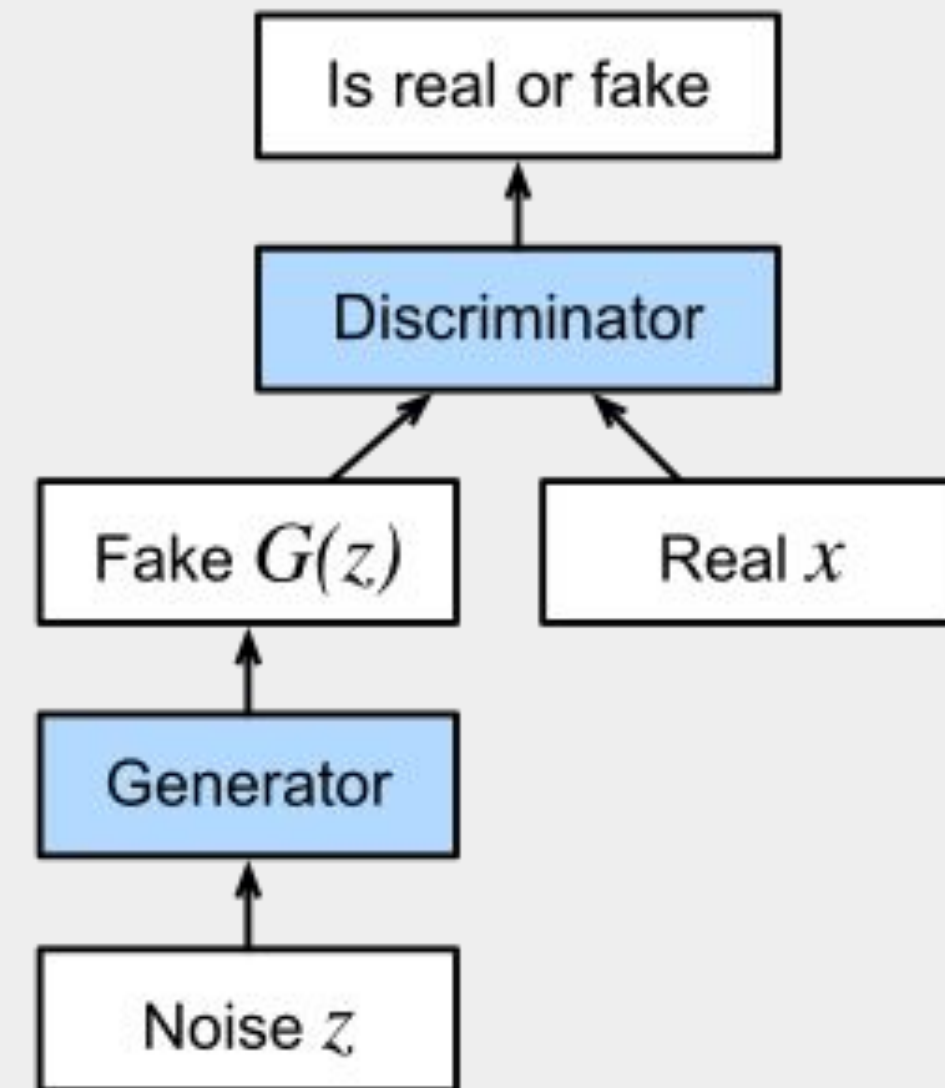
Blurry Output Quality: The most common complaint about VAEs is that their generated images are often blurry or lack fine detail. The reconstruction loss encourages the model to find an "average" of the data rather than sharp, specific details.

Posterior Collapse occurs when the model ignores the latent space during training. The encoder may learn a distribution that is too similar to the prior, causing the latent variables to lose meaningful information about the input.

Difficulty with Complex Data Distributions: VAEs rely on a predefined prior distribution (usually Gaussian) to organize their latent space. If the actual data has a highly complex or multi-modal distribution, a simple Gaussian prior may not be able to represent it accurately.

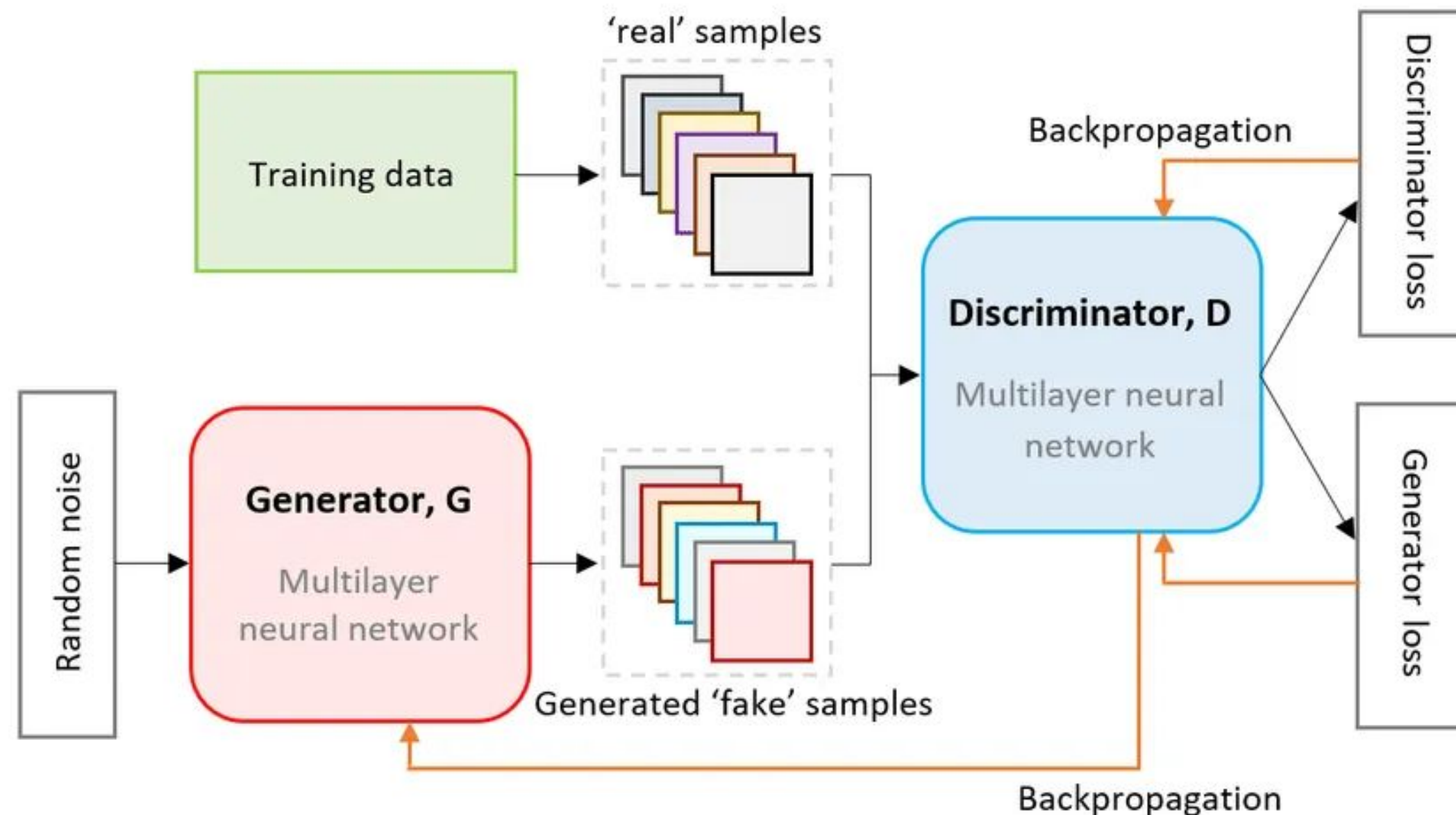


Generative Adversarial Networks



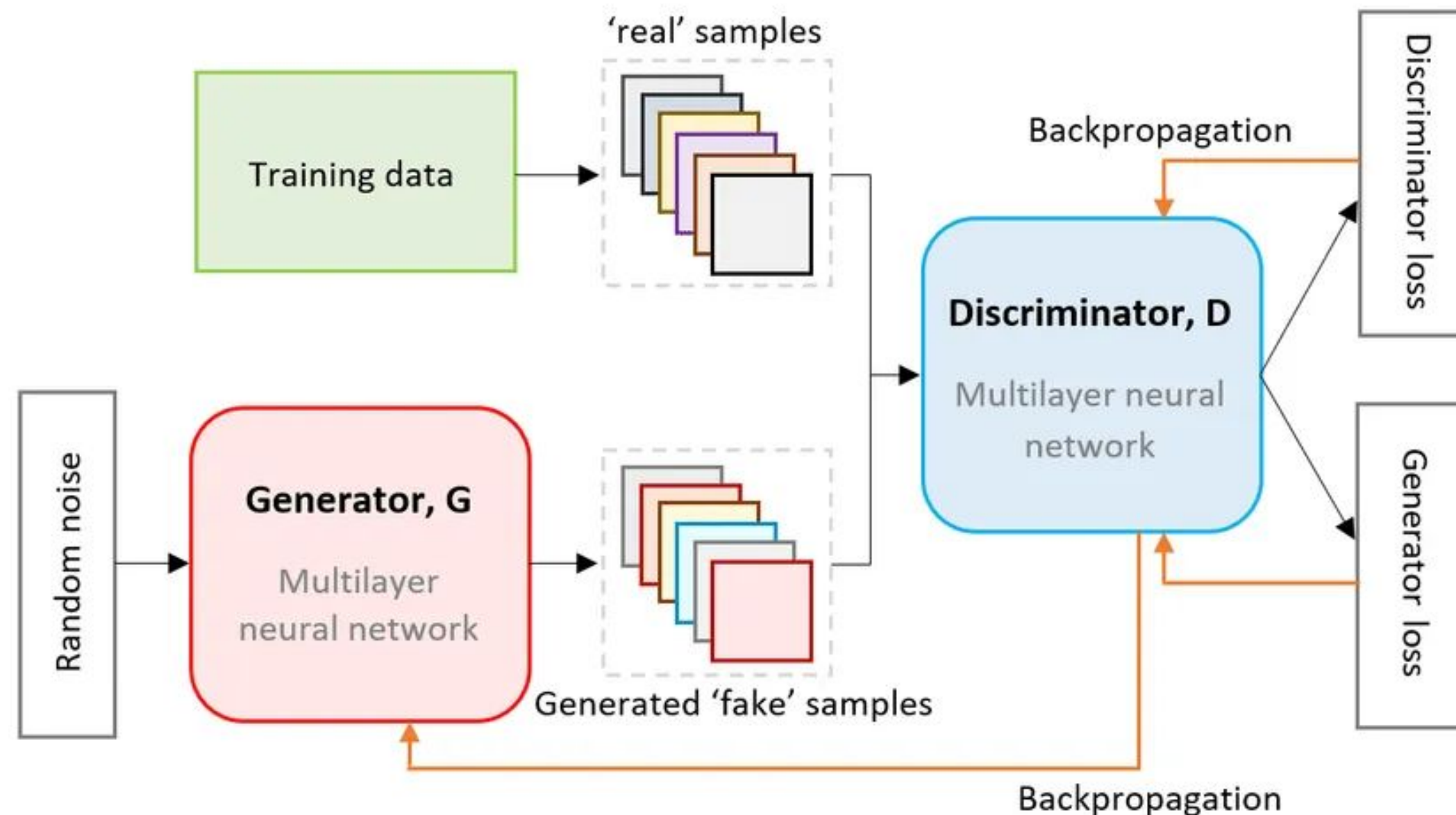
Generative Adversarial Networks

GANs learn the underlying probability distribution of complex, high-dimensional data without explicitly calculating the probability density function. It is introduced another network discriminator to represent the distribution difference between estimated and real data.



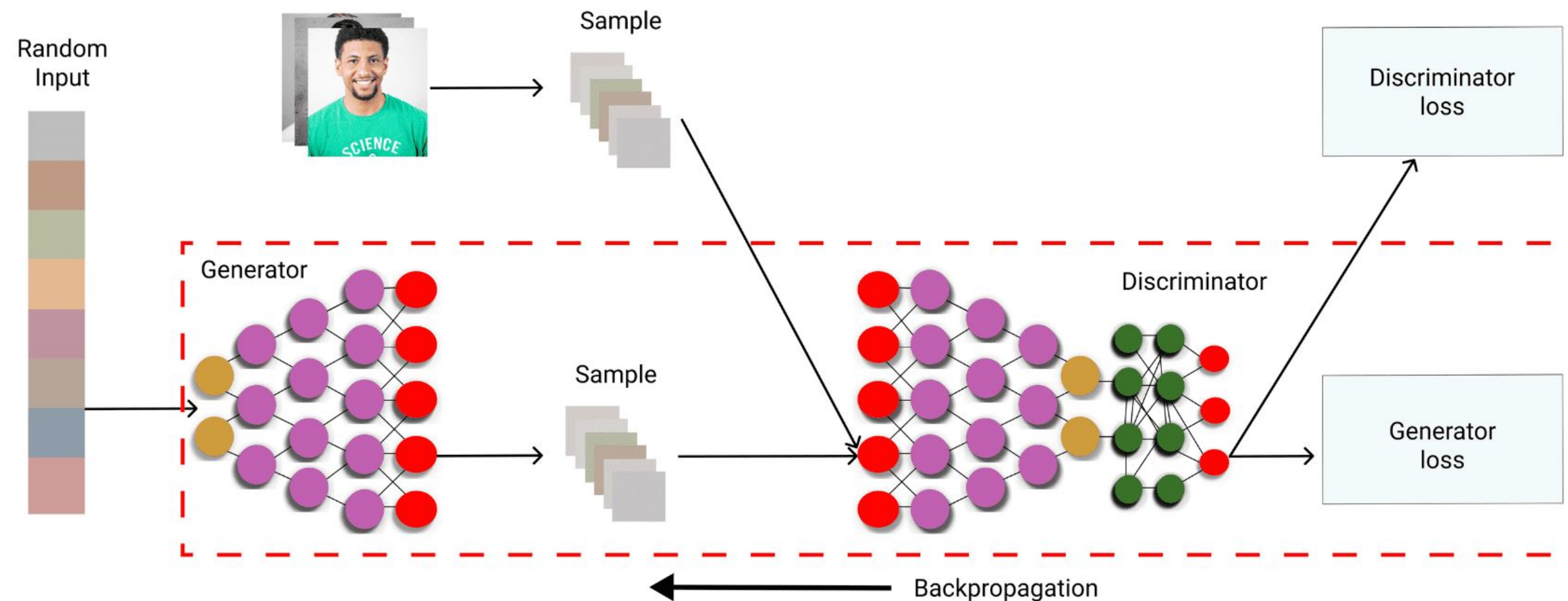
Generative Adversarial Networks

The **generator** works on the concept of the convolutional neural network, and the **discriminator** on the concept of the deconvolutional neural network. The generator produces outputs that are realistic looking and the discriminator identifies the fake ones from the real ones.



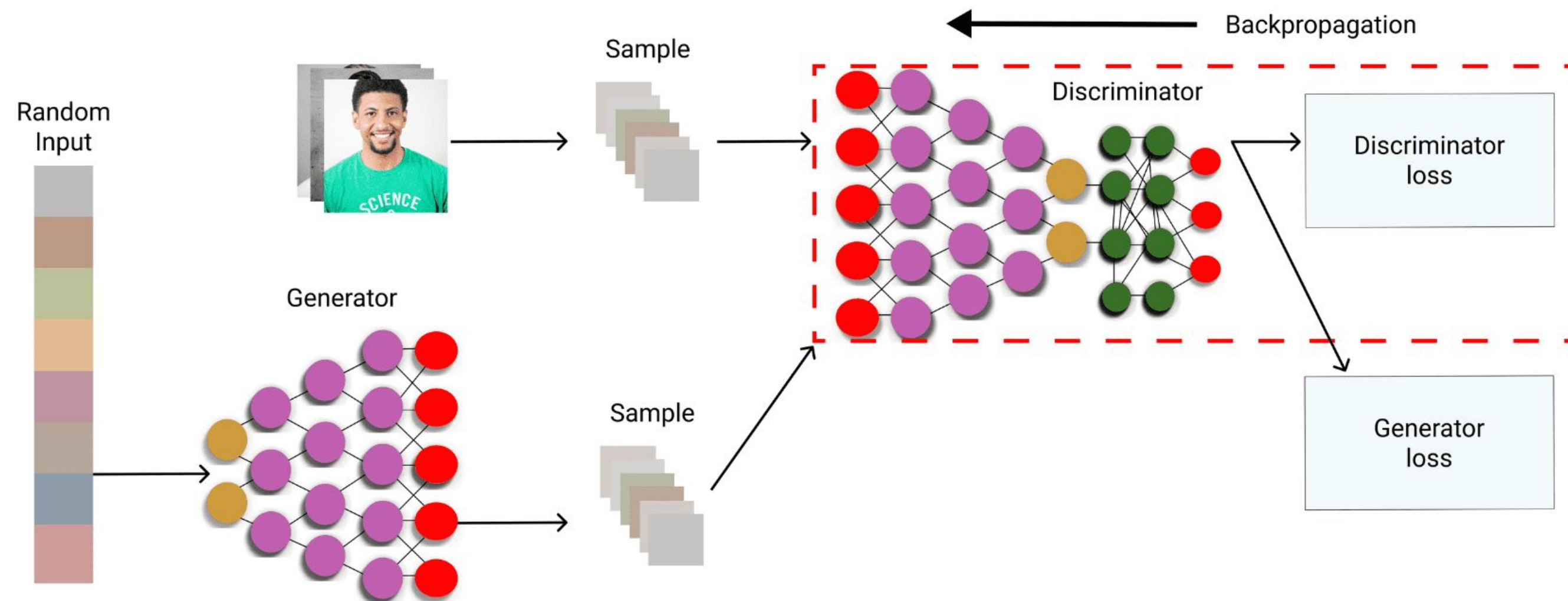
The Generator

A generator takes a random noise vector z from a latent space and map it to the data space, producing a synthetic output. It never actually sees the real training data. It's only feedback comes from its opponent.



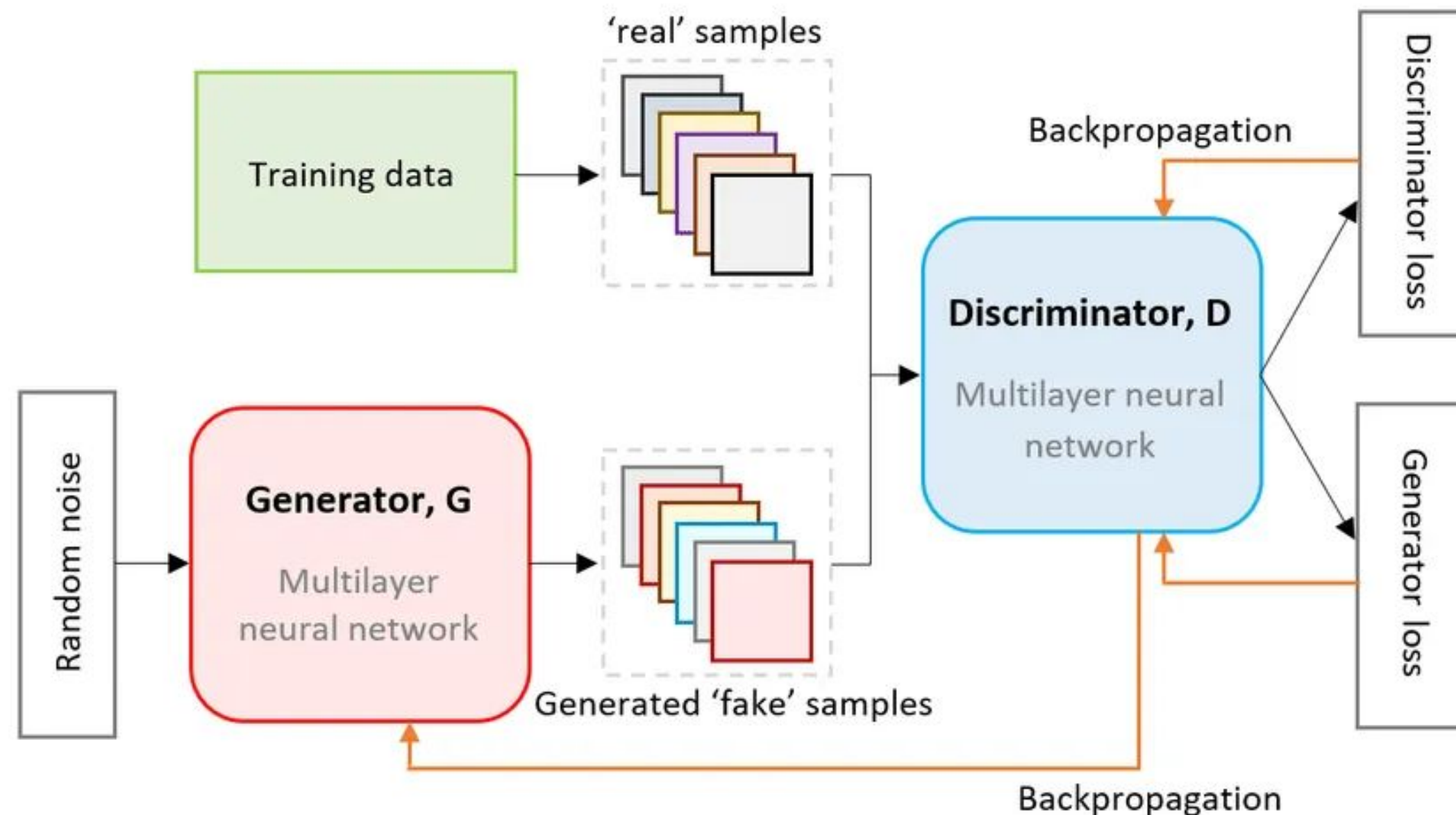
The Discriminator

A discriminator tries to evaluate the output generated by the generator with the original sample and gives a value between 0 and 1 as output. If the value is close to 0, then the generated sample is actually fake, and if the value is close to 1 then the generated sample is considered real.



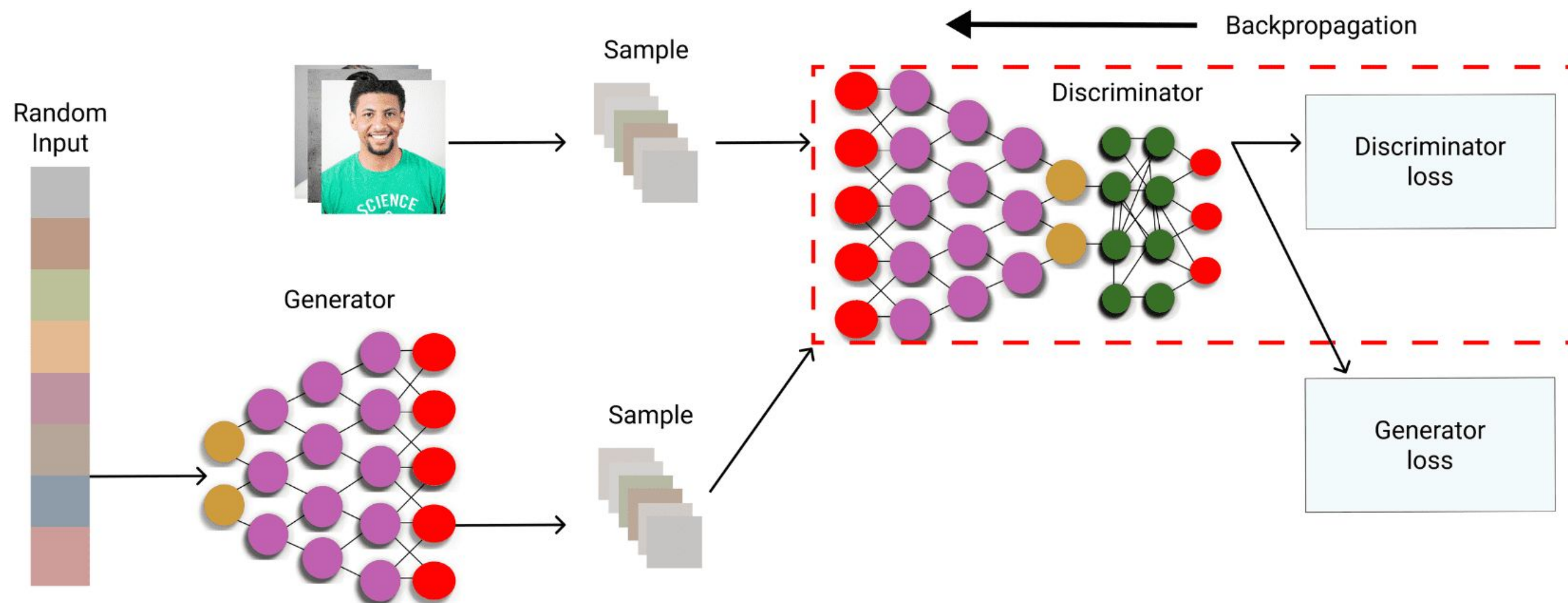
The Two-Phase Training Cycle

Training a Generative Adversarial Network (GAN) involves an alternating process where only one network is updated at a time. When training the generator, the discriminator must be "frozen" (its weights are held constant) and the other way around.



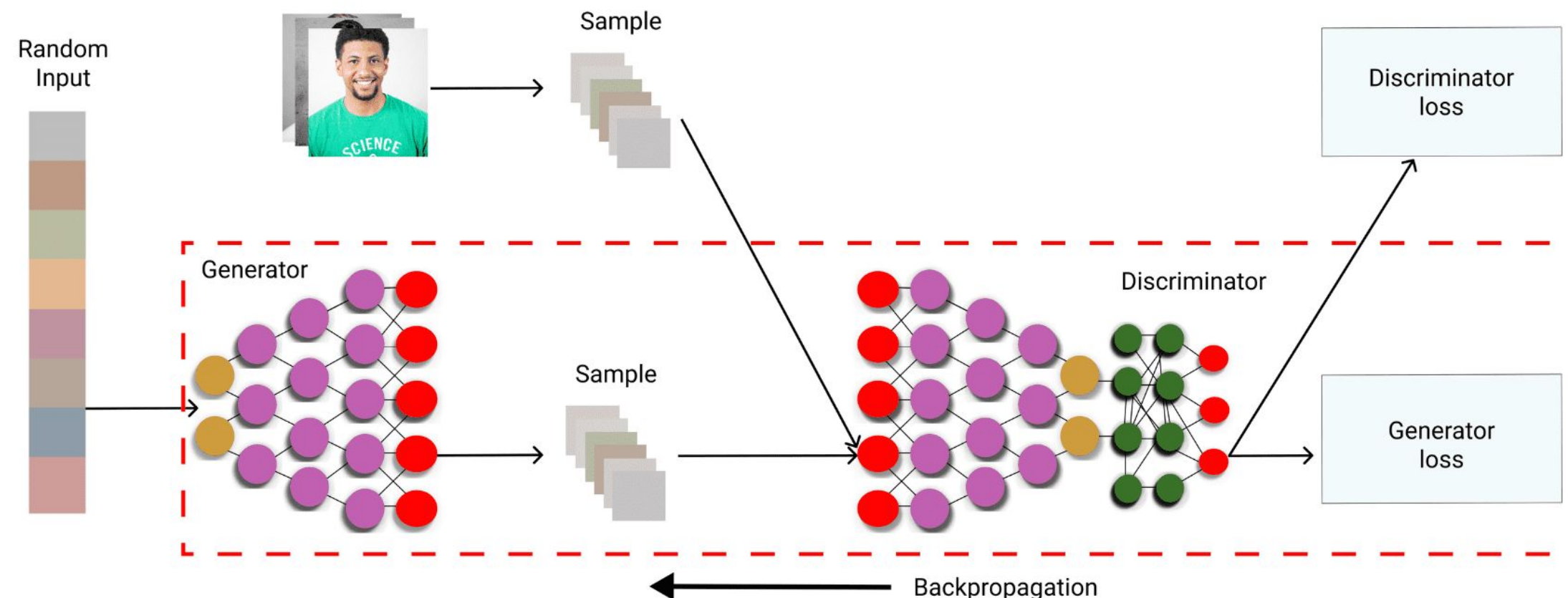
Phase 1: Update Discriminator (Generator Frozen)

1. The discriminator is fed both real samples from the dataset and fake samples from the generator.
2. During this phase, the generator's weights are constant; it simply acts as a source of "fake" data.
3. The discriminator learns to better distinguish between the two by updating its weights via backpropagation.



Phase 2: Update Generator (Discriminator Frozen)

1. The generator produces a batch of fake images.
2. These images are passed through the frozen discriminator to get a "real vs. fake" score.
3. Backpropagation flows through the entire system: it starts at the discriminator output, moves backward through the discriminator layers, and finally into the generator.
4. Only the generator's weights are updated using the resulting gradients. The discriminator weights remain unchanged so its classification standards stay consistent for that round.



What do we optimize for - The Nash Equilibrium

Unlike a standard neural network that simply rolls down a gradient hill **to a global minimum**, a GAN is searching for a **Nash Equilibrium**.

In game theory, this is the point where neither player can improve their position by changing their strategy alone.

For a perfectly trained GAN, the Generator learns to perfectly capture the true data distribution. At this exact point, the Discriminator **is completely guessing**; it cannot tell the difference between real and fake, and its output becomes exactly 0.5 for every input.

When we reach this point, we discard the Discriminator and keep the Generator to create ultra-realistic data!

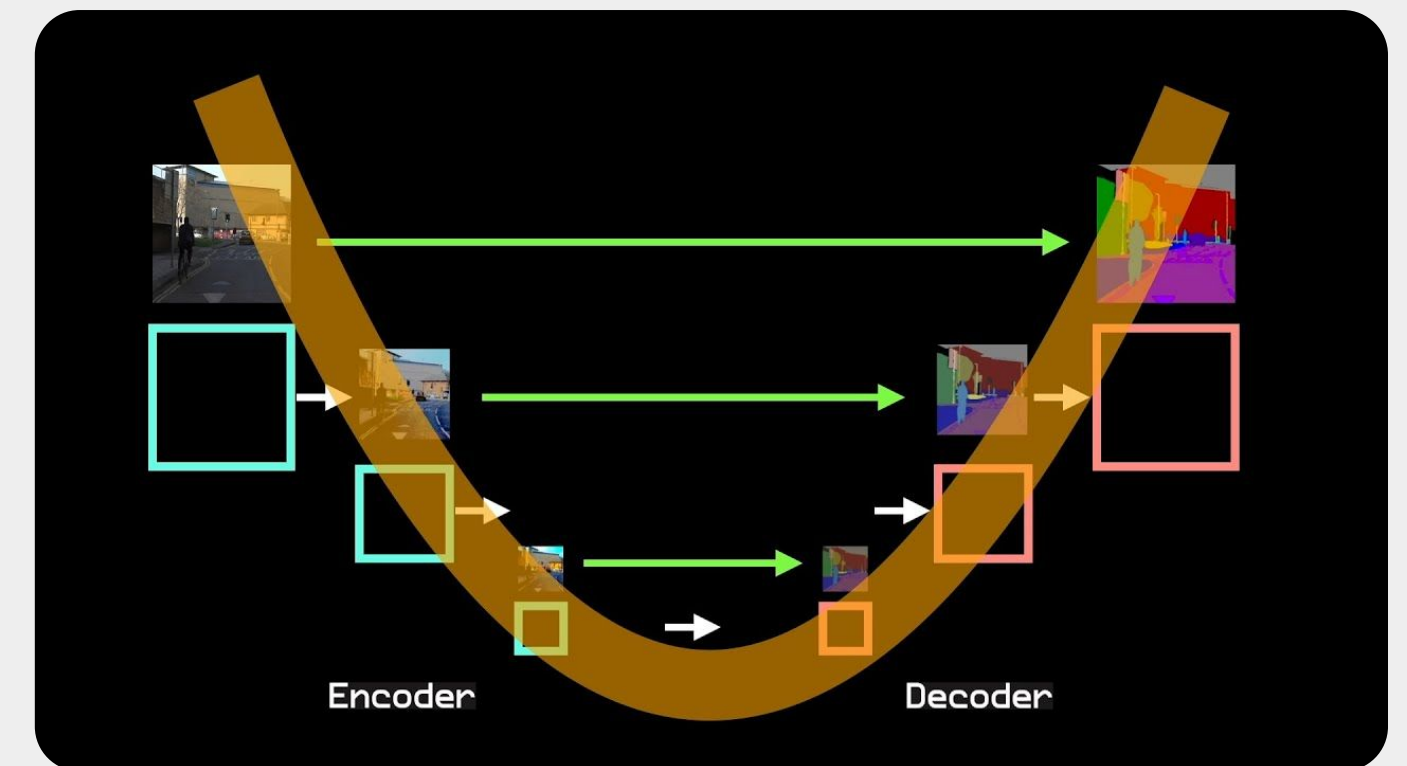
Mode Collapse: The Generator discovers a few "safe" outputs that fool the Discriminator and exclusively produces those, failing to capture the full diversity of the training data.

Training Instability (Non-Convergence): The zero-sum minimax game prevents standard gradient descent convergence. Network weights often oscillate dynamically instead of settling at a stable global minimum.

Vanishing Gradients: If the Discriminator becomes too accurate too quickly, its gradients drop to near zero. This cuts off the learning signal, permanently halting the Generator's progress.

The Discrete Data Bottleneck: GANs struggle with text and other discrete data. Generating text requires non-differentiable operations (like selecting a word from a vocabulary), which breaks the continuous backpropagation chain.

U-Net



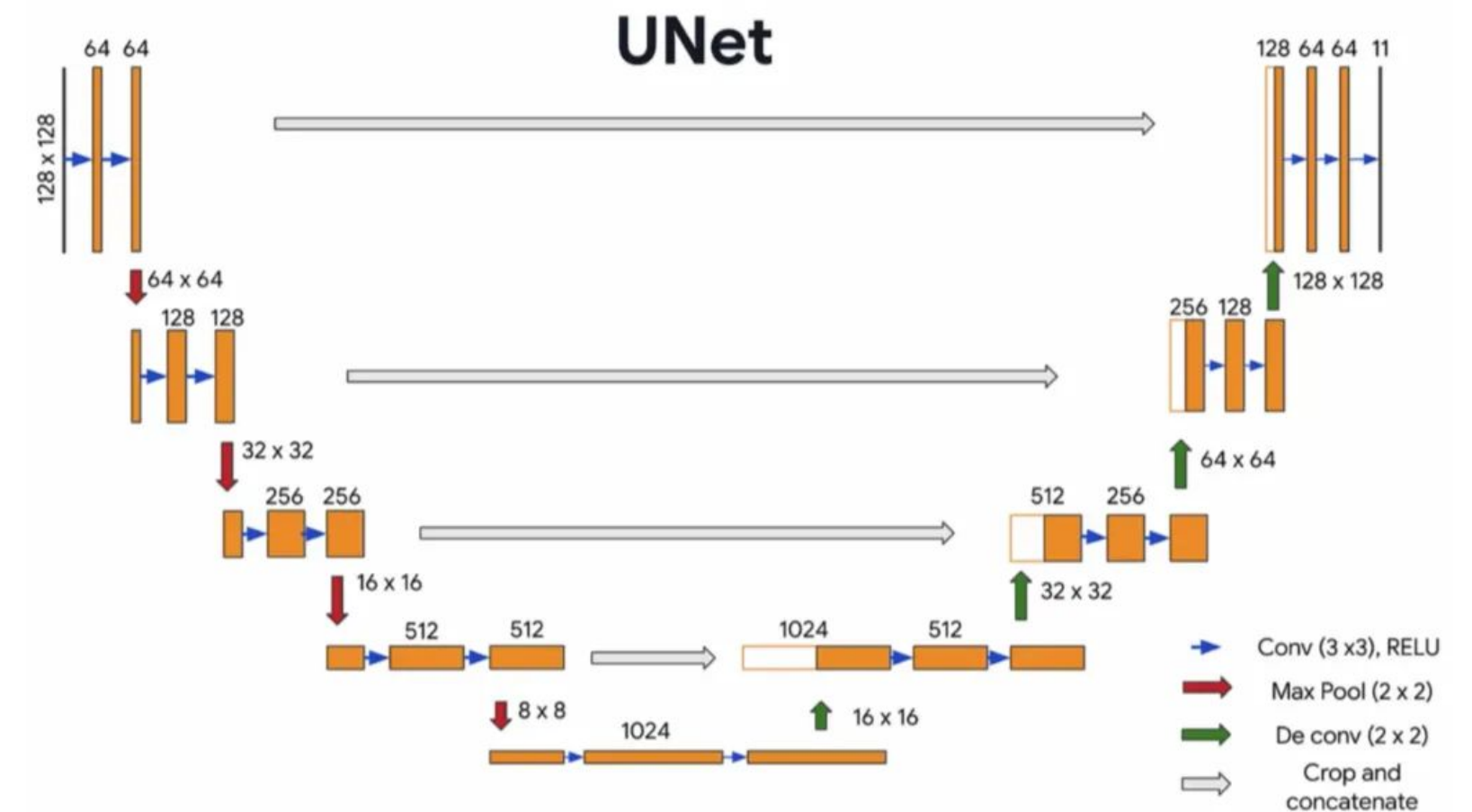
U-Net Architecture

U-Net was originally designed for image segmentation with many applications in biomedical imaging.

U-Net is a convolutional neural network (CNN) that follows an encoder-decoder structure.

Encoder (Contracting Path): This path captures context and downsamples the input image using convolutional and pooling layers, extracting high-level features.

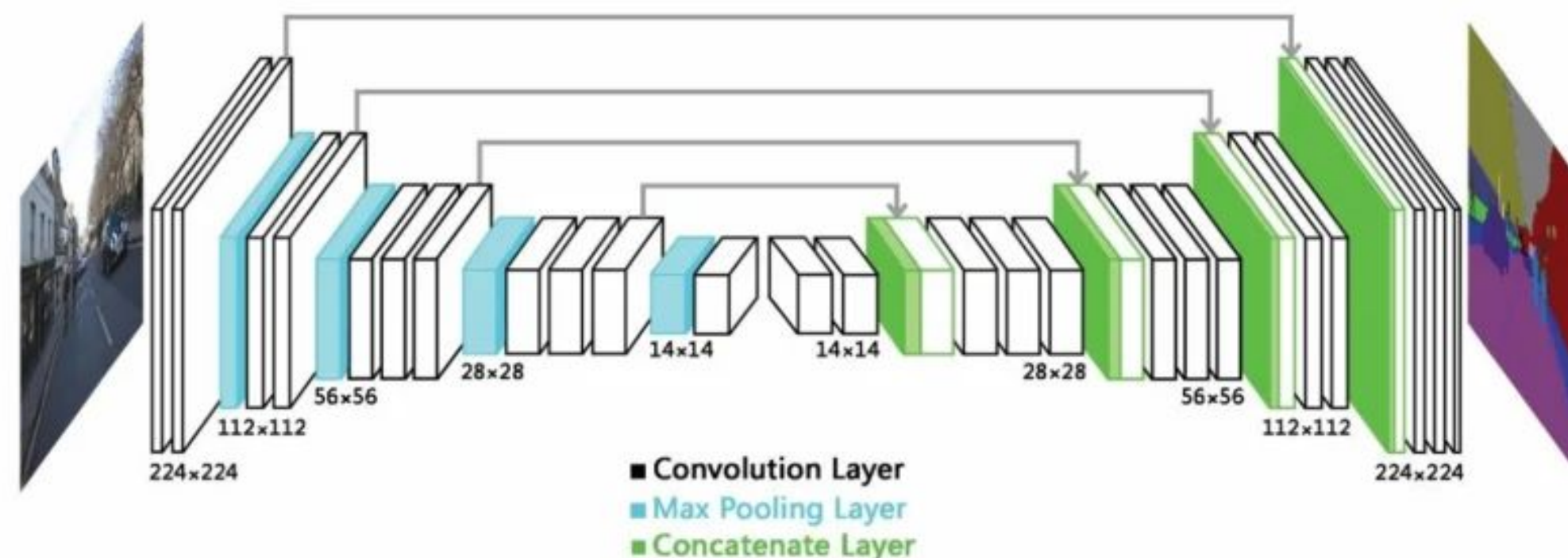
Decoder (Expanding Path): This path upsamples the features back to the original image size, enabling precise localization through transposed convolutions.



U-Net Architecture

What sets U-Net apart are the **skip connections** between the encoder and decoder layers. These connections concatenate feature maps from the contracting path to the corresponding expanding path layers, allowing the model to leverage both low-level and high-level features for better localization.

If U-Net only used the encoder and decoder, the final output would be blurry and useless. The spatial precision was permanently lost during the max pooling steps of the encoder, and the decoder cannot magically guess where the exact pixel boundaries used to be.



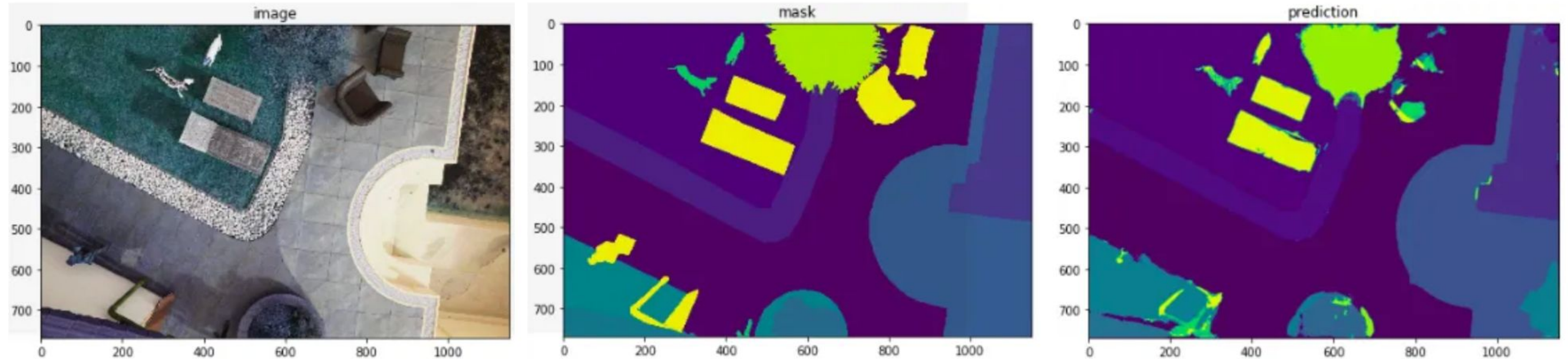
Bridging the Gap: At every single spatial resolution tier, U-Net takes the high-resolution feature maps from the encoder and passes them straight across the network to the decoder.

Concatenation over Addition: These skip connections are directly *concatenated* (stacked as new channels) onto the upsampled feature maps in the decoder.

Perfect Localization: As the decoder reconstructs the image, it now has access to both deep semantic context (from the bottleneck below) and raw, high-resolution spatial coordinates (from the skip connections). This allows the network to assemble pixel-perfect outputs.

Semantic segmentation

The network must assign a specific class label to every single pixel (e.g., outlining the exact boundaries of an object)



U-Net advantages and limitations

Data Efficiency: It can be trained effectively with very few annotated images (often just a few hundred), making it ideal for medical fields where expert labeling is expensive.

Speed: It is highly efficient; a 512x512 image can be segmented in less than a second on a GPU. Great for autonomous drive.

Diffusion Models: U-Net is the standard architecture used inside Diffusion models to predict and subtract noise (Denoising Engine). Also great for Autonomous Driving, Geospatial Analysis applications

Limitations

High VRAM Consumption: Concatenating skip connections doubles the feature map channels, requiring massive GPU memory and severely restricting batch sizes.

Lack of Global Context: Purely convolutional layers have limited receptive fields, making it difficult to capture long-range spatial dependencies across the image without injecting attention layers.

Thank you